MDPI

Article

An Adaptive Grouping Genetic Algorithm with Controlled Gene Transmission based on Fullness and Item Strategies (AGGA-CGT-FIS)

Stephanie Amador-Larrea [1]*, Marcela Quiroz-Castellanos [1], Octavio Ramos-Figueroa [1], and Alejandro Guerra-Hernández [1]

[1] Universidad Veracruzana, Artificial Intelligence Research Institute, Xalapa, México; oivatco.rafo@gmail.com(O. R.-F.), and aguerra@uv.mx(A. G.-H.)
* Correspondence: stephanieamadorlarrea@gmail.com, maquiroz@uv.mx

Abstract

The one-dimensional Bin Packing Problem (1D-BPP) is a well-known NP-hard grouping problem characterized by high structural complexity and broad practical relevance. Among the metaheuristic approaches proposed for this problem, the Grouping Genetic Algorithm with Controlled Gene Transmission (GGA-CGT) has shown remarkable performance. In this work, an Adaptive Grouping Genetic Algorithm with Controlled Gene Transmission based on Fullness and Item Strategies (AGGA-CGT-FIS) is presented. This approach extends the original GGA-CGT by integrating domain guided crossover mechanisms and adaptive parameter control schemes. The proposed algorithm incorporates a novel gene-level crossover operator, termed Fullness–Items Gene-Level Crossover 1 (FI-GLX-1). This operator exploits structural information from the solutions through Fullness- and Item-based ordering and transmission strategies. In addition, adaptive control schemes are introduced for key evolutionary parameters associated with crossover and mutation. These mechanisms allow the algorithm to dynamically adjust its behavior according to feedback extracted from the search process, resulting in a fully adaptive variant of the GGA-CGT. The effectiveness of AGGA-CGT-FIS is evaluated using two benchmark sets for the 1D-BPP: the classic and the BPPv_u_c instances. The proposed approach is compared against the baseline GGA-CGT using the original Gene-Level Crossover (GLX) operator. Experimental results show improvements in solution quality and convergence behavior, supported by statistical analyses that confirm the significance of the observed performance differences.

Keywords: One dimensional Bin Packing Problem; Grouping Genetic Algorithm; Crossover operators; Parameter control schemes

Received:
Revised:
Accepted:
Published:

Citation: Amador-Larrea, S.; Quiroz-Castellanos, M.; Ramos-Figueroa, O.; Guerra-Hernández, A. An Adaptive Grouping Genetic Algorithm with Controlled Gene Transmission based on Fullness and Item Strategies (AGGA-CGT-FIS). *Journal Not Specified* **2025**, *1*, 0. https://doi.org/

Copyright: © 2026 by the authors. Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the Creative Commons Attri- bution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Over the last four decades, the 1D-BPP has become increasingly relevant as a fundamental combinatorial optimization problem for improving resource allocation and operational efficiency in logistics, industrial, and transportation domains. It can be defined as a grouping problem as follows. Given a set of n items indexed by $j = 1, \ldots, n$, each with weight w_j, and an unlimited number of identical bins of capacity c. In this way, the goal is to assign each item to a exactly one bin such that the total weight in each bin does not exceed c and the total number of bins used is minimized. The mathematical formulation of the 1D-BPP is defined by the model given in Equations (1)-(5). First, Equation (1) presents

the objective function, where z denotes the number of bins used and y_i is a binary decision variable equal to 1 if bin i is used, and 0 otherwise.

$$\text{minimize } z = \sum_{i=1}^{n} y_i, \tag{1}$$

Additionally, the model is subject to the constraints defined in Equations (2)–(5). The first Constraint, in Equation 3), ensures that the total weight of the items assigned to each bin i does not exceed its capacity c, while the second Constraint, in Equations (3)–(5), guarantee that each item j is assigned to exactly one bin. The binary decision variable x_{ij} takes value 1 if item j is assigned to bin i, and 0 otherwise [1].

$$\sum_{j=1}^{n} w_j x_{ij} \leq c y_i, \quad i \in N = \{1, \ldots, n\}, \tag{2}$$

$$\sum_{i=1}^{n} x_{ij} = 1, \qquad j \in N = \{1, \ldots, n\}, \tag{3}$$

$$y_i \in \{0, 1\}, \qquad i \in N = \{1, \ldots, n\}, \tag{4}$$

$$x_{ij} \in \{0, 1\}, \qquad i, j \in N = \{1, \ldots, n\}. \tag{5}$$

This problem belongs to the NP-hard complexity class; therefore, solving it requires a high computational cost. As a result, it has been addressed using metaheuristic algorithms that offer high-quality solutions in short periods. Among these methods, the Grouping Genetic Algorithm (GGA), proposed in 1996 by Falkenauer et al. [2], is one of the most widely used [3]. This GGA is characterized by three key components: (1) a representation scheme in which groups are viewed as genes, (2) variation operators appropriated to the group-based representation scheme, and (3) a new fitness function that evaluates the utilization of bin filling.

Given the relevance of the GGA in solving grouping problems, subsequent research has aimed to strengthen its components. In 2015, Quiroz-Castellanos et al. [4] introduced the GGA-CGT, an enhanced version of Falkenauer's GGA [5] which exhibits superior performance. This improvement is attributed to the incorporation of search heuristics that promote the transmission of the best groups, controlling the balance between selective pressure and diversity. Building upon these advances, recent literature reports several GGA variants to address the 1D-BPP. One of the most recent studies is presented in [6], where the authors conducted an experimental study of group-oriented crossover operators for the GGA-CGT designed to solve the classical 1D-BPP benchmark [4]. They concluded that the GLX operator is the most suitable in terms of the number of optimal solutions found. Nonetheless, they also observed that this operator does not significantly impact the final algorithm's performance. The state-of-the-art also reports numerous successful examples of variation operator design for GGAs and grouping problems, primarily crossover and mutation, based on experimental methodologies that extract problem-domain knowledge and analyze algorithmic behavior, such as the approach presented in [7]. For the case of mutation operators, these studies include the 1D-BPP [8], the unrelated parallel machine scheduling problem with makespan minimization (R||Cmax) [9], the Image Segmentation (IS) problem [10], and Variable Decomposition for Large-Scale Constrained Optimization Problems (VD-LSCOP) [11].

Likewise, other works have concentrated on the design of crossover operators in problem domains, like 1D-BPP [4], R||Cmax [7], IS [12], and VD-LSCOP [10]. Beyond

operator design, recent research has increasingly emphasized the exploitation of the intrinsic dynamism of the GGA search process through parameter control schemes, enabling adaptive behavior in response to the evolving state of the search. Notable applications of such control mechanisms within GGAs include 1D-BPP, R||Cmax, and VD-LSCOP [13,14].

Motivated by these findings, this work presents a comprehensive experimental study of group-oriented crossover operators for the GGA-CGT, using a unified benchmark that combines the newly introduced and challenging BPPv_uc with the classical test instances. The study is structured into eight phases designed to systematically enhance and validate the performance of the state-of-the-art GGA-CGT. The first phase evaluates the performance of the original GGA-CGT using several state-of-the-art grouping crossover operators. The subsequent two phases investigate alternative strategies for sorting parental genes and regulating gene transmission. The fourth phase focuses on analyzing and understanding the behavior of GGA-CGT with the implemented improvements, using a set of indicators to characterize the search process and descriptive visualizations. Next, an adaptive control scheme for the n_c, which governs the number of individuals used in the crossover process, is introduced and its impact is examined. In the seventh phase, the effectiveness of a novel repair strategy designed to reinsert items lost during the mating process is assessed. Finally, the last phase incorporates additional adaptive schemes to regulate parameters associated with the mutation operator.

The main contributions derived from this experimental study are summarized as follows: (1) an experimental approach for developing grouping crossover operators through the generation and analysis of both problem-domain characteristics and algorithmic behavior within the context of the GGA-CGT applied to the 1D-BPP; (2) the definition of a set of indicators to characterize and analyze the crossover process; (3) the proposal of the FI-GLX-1 operator, which incorporates strategies for sorting parental genes and controlling the genetic material transmission based on solution structure, i.e., the bin fullness and the number of items per bin; and (4) the proposal of the AGGA-CGT-FIS, an adaptive extension of GGA-CGT that integrates parameter control schemes for crossover and mutation parameters, as well as solution structure-based strategies to regulate the crossover process.

The remainder of this paper is organized as follows. Section 2 reviews recent advances in solving the 1D-BPP. Section 3 introduces the GGA-CGT and describes its main components. Section 4 surveys state-of-the-art grouping crossover operators for the group-based representation considered in this work. Section 5 details the multi-phase experimental approach adopted to design the proposed AGGA-CGT-FIS. Section 6 presents the experimental results and statistical analyses conducted to evaluate the effectiveness and significance of the proposed algorithm. Finally, Section 7 concludes the paper and outlines directions for future research.

2. Recent Advances in Solving the 1D-BPP

From an algorithmic perspective, the 1D-BPP has been addressed using a wide variety of solution methodologies, including approximation algorithms, exact methods, neighborhood searches, evolutionary algorithms, swarm intelligence algorithms as well as hyper heuristics and hybrid methods. Moreover, some studies have analyzed the characteristics of the 1D-BPP instances, generating subsets of features to discriminate between problem instances with different attributes. These features have been used for the design of specialized methods, focused on solving classes of instances, as well as for the selection and configuration of strategies in sophisticated hybrid methods that incorporate adaptive approaches and advanced machine learning techniques.

Heuristic methods for the 1D-BPP are traditionally categorized as item-oriented or bin-oriented. The first group considers items one by one (given some initial order) and

packs them in one of the open bins, opening new bins when necessary. The second group attempts, for each bin, to find a set of items (packing) that maximizes capacity utilization [15]. Although no polynomial-time approximation algorithm for the 1D-BPP is known to achieve a worst-case performance ratio smaller than 3/2 unless P=NP [16], such heuristic methods are commonly used to provide an initial solution to exact and metaheuristic algorithms, as well as to modify and rebuild solutions within operators and strategies to traverse the search space.

Among exact methods, the state-of-the-art approaches are based on pseudopolynomial formulations [17] and branch-price-and-cut frameworks [18,19]. Although these algorithms have shown highly competitive performance in finding optimal solutions in classical benchmark instances, there are still cases that have not been solved satisfactorily, even for problems of moderate size. As a result, increasing attention has been devoted to heuristic approaches such as neighborhood search, where state-of-the-art methods include tabu search and hill climbing. In particular, the hybrid heuristic HI-BP [20] combines lower bounds, dual solution generation, guided redistribution, and tabu search to obtain robust results. Another relevant approach is CNS_BP [21], which applies consistent local search on partial solutions through add/remove moves combined with tabu search and hill climbing. Beyond neighborhood-based heuristics, evolutionary algorithms have also been extensively explored for the 1D-BPP. Within this context, the GGA-CGT [4] and its improved version GGA-CGT/D [22] are among the most competitive approaches, incorporating lower-bound selection strategies, problem reduction techniques, and mechanisms to promote population diversity. Regarding swarm intelligence algorithms, recent proposals incorporate ant colony optimization and flower pollination algorithms, however, their performance still falls far short of the state-of-the-art methods [23,24]. The comparative study of [22], which analyzes twenty-one metaheuristics for the 1D-BPP, confirms that CNS_BP and GGA-CGT are consistently among the top-performing methods on the classical benchmark set. These results highlight the effectiveness of advanced metaheuristic frameworks for the problem, while also suggesting that further performance gains may depend on adaptive and higher-level decision mechanisms, rather than on isolated algorithmic components.

In response to the need for adaptive and higher-level decision mechanisms, hyper-heuristics, heuristics to choose heuristics, have emerged as a promising approach to finding good solutions to 1D-BPP. A competitive proposal combines a simulated annealing acceptance criterion, a stochastic heuristic selection strategy, and a short-term memory [25]. Recently, a micro genetic algorithm to generate selection hyper-heuristics was proposed, in which the problem state representation is enriched with features that characterize 1D-BPP instances [26]. Although these approaches have yielded good performance across several data sets and their design has provided valuable insights about the heuristics behavior and its relationship with the characteristics of 1D-BPP instances, their results are still not comparable to those of the state-of-the-art methods.

Furthermore, sophisticated hybrid methods have been proposed to enhance performance by explicitly exploiting instance characteristics. Among them, Perturbation-SAWMBS [15] is one of the most prominent methods, controlling the average weight of items per bin and integrating constructive, improvement, and reduction heuristics tailored to the 1D-BPP. Recently, an algorithm was proposed to solve the 80 Falkenauer T problem instances (Triplets), which can operate either using backtrack or local search to find optimal packing [27]. While these methods have shown good results for instances with certain properties, their performance is usually poor when considering sets of instances with different features.

Latest research has increasingly focused on internal mechanisms that influence algorithmic performance. Regarding parameter control, numerous adaptive schemes have

been proposed for genetic algorithms to dynamically regulate numerical parameters associated with variation operators, including adaptive crossover-control rules [13,28–30]; mutation-control schemes [28,30–33]; and adaptive mechanisms for other variation operators [13,28,31,34–38]. Extending these ideas, an adaptive rule to regulate the mutation operator in the GGA-CGT was proposed in [14], enabling the use of disruptive strategies under low-diversity scenarios. Likewise, the authors of the study reported in [13] analyzed different adaptive control schemes for four parameters of three GGAs, including the GGA-CGT, for the crossover rate, the mutation rate, and two auxiliary mutation-related parameters. Both works reported moderate but consistent improvements when adaptive techniques were applied. However, their principal contribution lies in reducing the parameter-tuning complexity without compromising the competitiveness of the results.

The literature on the 1D-BPP is substantial, and great efforts have been carried out by proposing and adopting new approaches and strategies to obtain better solutions. While many classes of instances have been optimally solved by most of the state-of-the-art algorithms, there are still benchmarks that present a great difficulty. In such cases, it is unclear how to select and integrate the appropriate techniques to solve them and what is the expected performance of different strategies. Overall, the literature suggests that feedback-driven mechanisms can significantly enhance the exploration–exploitation balance and improve the robustness of evolutionary algorithms. From this standpoint, the present work introduces a control scheme specifically designed for the crossover operator, complemented by adaptive strategies for its related parameters and aligned with the trends identified in previous research.

3. GGA-CGT for 1D-BPP

Building upon the advances discussed above, this section presents a detailed description of the GGA-CGT, a high-performance GGA variant specifically designed to address the 1D-BPP. The GGA-CGT begins by generating an initial population using a heuristic initialization strategy. Subsequently, From this standpoint, is carried out over a finite number of generations. At each generation, the GGA-CGT iteratively applies parent selection, crossover, mutation, repair, and replacement mechanisms in order to progressively improve the quality of the solutions. In addition to the maximum number of generations, this algorithm incorporates a second stopping criterion based on the theoretical lower bound L_2 [1], which represents the closest attainable approximation to the optimal number of bins. Once a solution reaches this bound, no better feasible solution can be obtained.

Algorithm 1 presents the pseudocode of the GGA-CGT, providing a step-by-step description of its execution and highlighting the interaction between its main components, including selection, crossover, mutation, repair, and replacement mechanisms. This pseudocode provides an overview of the algorithmic flow, aiding the understanding of the operators and adaptive control strategies detailed in the following sections. To facilitate readability, the main parameters used throughout the experimental phases are defined as follows: $|P|$ denotes the population size, max_gen the maximum number of generations, n_c the number of individuals selected for crossover, and n_m the number of individuals selected for mutation.

3.1. Chromosome representation in the GGA-CGT

In the GGA-CGT, each individual encodes a complete solution to the 1D-BPP through the group-based chromosome representation. Unlike classical genetic algorithms that represent solutions as linear strings, the GGA-CGT employs a structured representation in which genes correspond to bins and each gene contains a group of items assigned to it.

Algorithm 1: GGA-CGT

Input: Maximum generations max_gen, stopping threshold L_2, population size $|P|$
Output: Best solution obtained $global_best_solution$
Generate the initial population P using FF-$\tilde{n}$;
while $generation < max_gen$ ***and*** $|best_solution| > L_2$ **do**
  Select n_c individuals using *Controlled_Selection*;
  Apply *GLX* to the n_c selected individuals;
  Apply *Controlled_Replacement* to introduce the offspring;
  Select n_m individuals using *Controlled_Selection* and clone elite solutions;
  Apply *Adaptive_Mutation+RP* to the best n_m individuals;
  Apply *Controlled_Replacement* to introduce the clones;
  Update $global_best_solution$;
end

Formally, a chromosome is defined as a set of m genes, where each gene represents a bin and stores the indices of its assigned items. Feasibility is ensured by construction, since the total weight of the items assigned to each bin does not exceed the bin capacity c, and each item appears in exactly one gene. Consequently, a chromosome represents a valid packing configuration composed of m bins.

This group-based representation is particularly suitable for the 1D-BPP, as it preserves meaningful structural information about how items are combined within bins. Moreover, it allows genetic operators to manipulate entire bins or subsets of bins, rather than individual items, which facilitates the transmission of high-quality building blocks during the evolutionary process.

Figure 1 provides an illustrative example of the chromosome representation used in the GGA-CGT. In this encoding, an individual X_i is composed of multiple genes, each corresponding to a bin that contains a subset of item indices. The total weight of the items assigned to each bin is shown below the corresponding gene, illustrating the degree of bin utilization while respecting the capacity constraint. The fitness value associated with the individual is displayed on the right-hand side, summarizing the overall quality of the packing configuration. This example is intended solely to illustrate the structure of a chromosome and its components.

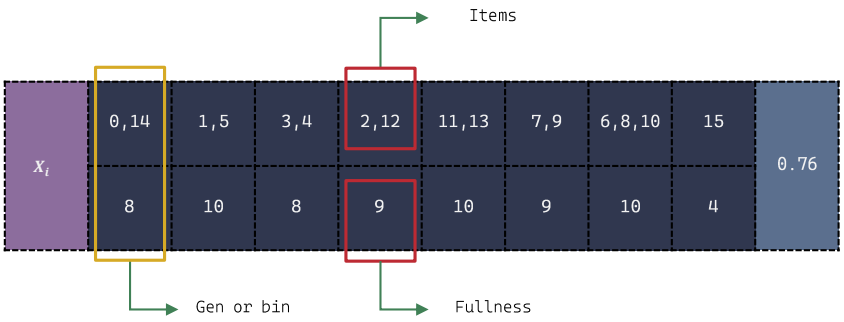


Figure 1. Illustrative example of a chromosome in the GGA-CGT corresponding to a Bin Packing Problem instance with 16 items and bin capacity $c = 10$. The set of items is $N = \{0, 1, \ldots, 15\}$, with associated weights $(1, 3, 4, 2, 6, 7, 5, 8, 3, 1, 2, 4, 5, 6, 7, 4)$. For each gene, the total weight of the assigned items is reported, illustrating the level of bin utilization under the capacity constraint.

Overall, this representation establishes a strong correspondence between the problem structure and the evolutionary search process, allowing the GGA-CGT to efficiently explore and exploit the solution space of the 1D-BPP.

3.2. Fitness function

The objective function is a key component of the GGA-CGT since it measures the solution's quality. In general terms, the GGA-CGT purpose is to maximize the population's solution quality. To achieve this goal, it uses the fitness function presented in Equation (6),

where m corresponds to the number of bins in the solution, S_i represents the sum of the weights of the items assigned to the bin i, and c denotes the capacity of the bins.

$$F_{BPP} = \Sigma_{i=1}^{m} \frac{(S_i/c)^2}{m}. \tag{6}$$

Although the 1D-BPP is formally stated as the minimization of the number of used bins (as shown in Equation (1)), the GGA-CGT is driven by a maximization criterion. For this reason, the original objective is operationalized through the fitness function defined in Equation (6), which evaluates the quality of a solution by measuring how tightly the bins are filled. The term $(S_i/c)^2$ assigns higher contributions to bins that are more efficiently filled, while penalizing poorly filled bins. Consequently, maximizing F_{BPP} promotes more compact packings and reduces wasted space. In practice, solutions exhibiting more compact bin utilization tend to require fewer bins, thereby aligning the fitness maximization with the original minimization objective. Therefore, the fitness function in Equation (6) provides a smooth and discriminative measure of solution quality that effectively guides the evolutionary search, even when multiple solutions use the same number of bins.

3.3. Initial population

The population initialization strategy is responsible for establishing the starting point of the search. Depending on the problem search space, the performance of this strategy can strongly influence the GGA-CGT final results. When suitable heuristics are employed, the initial population can be close to the best-known solution, facilitating efficient convergence. The GGA-CGT uses the FF-ñ population initialization heuristic. This strategy begins by arranging in different bins the $\tilde{n}$ items with a weight greater than 50 percent of the bin capacity c. Then, it randomly permutes the remaining items using a uniform distribution. Finally, the permuted items are assigned to bins according to the classical First Fit (FF) heuristic, where each item is placed in the first bin with sufficient remaining capacity; if no such bin exists, a new bin is opened [1].

3.4. Grouping crossover operator

GGA-CGT works with the Gene-Level Crossover (GLX) operator, which generates two offspring from a pair of parent solutions as follows. First, both parents are sorted in descending order concerning the fill level of each gene (i.e., bin). Then, the genes from both parents are compared in parallel, and the gene corresponding to the parent with the higher filling level is inherited by the offspring, followed by the gene from the other parent. In a tie, the first child gives priority to the first parent, and the second child gives priority to the second parent.

3.5. Grouping mutation operator

Regarding the mutation, GGA-CGT uses the Adaptive Mutation operator to remove part of the m bins in some solutions. The number of bins to be removed is given by the relation between the size of the solution and the number of incomplete bins, i.e. those that have not reached their maximum capacity. The number of bins to remove is computed as follows:

$$n_B = [\imath \cdot \epsilon \cdot p_\epsilon], \tag{7}$$

where $\imath$ corresponds to the incomplete bins of the solution, ϵ the elimination portion, and p_ϵ the probability of elimination.

$$\epsilon = \frac{(2 - (\imath/m))}{\imath^{(1/k)}}, \tag{8}$$

$$p_\epsilon = 1 - U(0, \frac{1}{\iota^{1/k}})). \tag{9}$$

The parameter k ($k > 0$) defines the rate of change of ϵ and p_ϵ. The elimination ratio is inversely proportional to the number of incomplete bins ι and the percentage of incomplete bins ι/m.

3.6. Rearrangement by pairs

To restore feasibility and preserve solution quality, the repair strategy Rearrangement by Pairs (RP) is applied. This strategy operates in two stages and aims to improve both the packing structure and the subsequent reinsertion of free items.

In the first stage, RP scans the bins of the current solution in a randomized order and attempts to perform local exchanges between packed and free items. For a given bin, the method examines pairs of packed items and evaluates candidate replacements using either one free item or a pair of free items. The replacement is accepted only if two conditions are simultaneously satisfied. First, it is ensured that the capacity constraint of the bin is not violated. Subsequently, it is verified that the replacement leads to an improvement in bin utilization, such that the total weight of the incoming item or item pair is greater than or equal to the combined weight of the removed packed pair. Once a feasible exchange is found for a bin, it is immediately applied and the procedure moves on to the next bin, avoiding excessive modifications of the same bin.

In the second stage, the remaining free items are reinserted using the FF-$\tilde{n}$ heuristic. This step completes the solution and allows the algorithm to exploit the rearranged packing structure produced in the first stage. The use of randomized orders in the first stage promotes diversification by reducing deterministic biases, while the weight-based replacement rule encourages the formation of fuller bins, which improves the quality of the genetic material used in subsequent generations. Algorithm 2 summarizes the RP procedure.

Algorithm 2: RP

Input: Current solution S, set of free items F
Generate a random permutation of the bins in S, obtaining a sequence of b bins $S = (B_1, \ldots, B_b)$;
Generate a random permutation of the free items in F;
$j \leftarrow 1$;
while $j \leq b$ **do**
  foreach *pair of items* (p, s) *in bin* B_j **do**
    exchanged $\leftarrow$ FALSE;
    foreach *pair of items* (i, k) *in* F **do**
      if $w_i \geq w_p + w_s$ ***and*** *Feasible*$(S' \leftarrow Swap(S, F, (p, s), i))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        exchanged $\leftarrow$ TRUE;
        break;
      end
      if $w_k \geq w_p + w_s$ ***and*** *Feasible*$(S' \leftarrow Swap(S, F, (p, s), k))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        exchanged $\leftarrow$ TRUE;
        break;
      end
      if $w_i + w_k \geq w_p + w_s$ ***and*** *Feasible*$(S' \leftarrow Swap(S, F, (p, s), (i, k)))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        exchanged $\leftarrow$ TRUE;
        break;
      end
    end
    if *exchanged* **then**
      continue;
    end
  end
end
Apply FFD to reinsert remaining free items in F into solution S;

3.7. Selection mechanism

The variation operators work hand in hand with the selection mechanisms, since they determine the criteria for choosing the solutions to be modified with each operator in each generation. GGA-CGT uses the controlled selection scheme. For the crossover operator, this mechanism selects n_c parents to generate n_c offspring as follows. First, two sets of parents G and R are generated, G with $n_c/2$ individuals randomly selected with a uniform distribution from the best n_c individuals in the population, and R with $n_c/2$ individuals randomly chosen with a uniform distribution from the population without repeating and discarding the individuals from the elite group (a group of individuals with the best fitness that remains unchanged for a number of generations). Thus, each pair of parents i includes one individual G_i taken from G and one more R_i from R $(0 \leq i < (n_c)/2)$. On the other hand, the individuals to mutate are selected in two stages, as follows. First, individuals in the elite group that still have a remaining elite lifetime are cloned. Then, the best n_m solutions are mutated.

3.8. Replacement mechanism

As in the selection mechanism, the procedures of the variation operators are related to the replacement mechanisms, which also operate in a controlled manner. This mechanism is applied to both offspring generated through crossover and cloned individuals. For the crossover, the n_c offspring are incorporated into the population using two criteria. First, $n_c/2$ children replace the set of random parents R. The remaining $n_c/2$ children replace individuals with repeated fitness, followed by those with the lowest fitness. Similarly, cloned solutions first replace individuals exhibiting duplicate fitness values and subsequently those with the lowest fitness [4].

4. Grouping crossover operators

Variation operators are a fundamental component of GGAs, as they determine the search dynamics by generating new candidate solutions. The literature reports a wide variety of genetic operators tailored for grouping problems under different structural constraints, including mutation, translocation, injection, and cloning. Among these, the crossover, also called the mating operator, is the most widely adopted [39]. An effective crossover operator should be able to extract high-quality structural traits from two or more parent solutions and recombine them to produce offspring with improved characteristics. Such operators enhance the exploitation capabilities of GGAs and enable a more focused search within promising regions of the solution space.

Given the central role of crossover in grouping problems, a variety of recombination strategies have been proposed in the literature, each characterize by distinct properties and design principles. This work focuses on a subset of crossover operators that have demonstrated strong performance and are particularly suitable for the 1D-BPP [6]. The operators considered are: Uniform Crossover (UX), Exon Shuffling Crossover (ESX), Greedy Partition Crossover (GPX), Gene-Level Crossover (GLX), and Gene-Level Crossover V1 (GLX-1). Table 1 summarizes the main characteristics of each crossover operator, including (1) the number of offspring generated, (2) whether parental genes are sorted prior to transmission, (3) whether probabilistic rules are used to control gene inheritance, and (4) whether each parent has the possibility of transmitting all its genetic material to both offspring.

4.1. Uniform Crossover

.

The first group-oriented crossover operator studied is UX, a simple operator that generates a pair of offspring C_1 and C_2 from two parent solutions P_1 and P_2. Its crossover process is as follows. It considers the parents in parallel concerning the genes. Thus, for each pair of genes in position i, this operator generates a random value r in (0,1) with a uniform distribution. If $r \leq 0.5$, the children C_1 and C_2 receive the genes in position i from P_1 and P_2, respectively. Otherwise, the first child C_1 receives the gene i from the second parent P_2 and the second child C_2 the gene i from the first parent P_1. This operator can generate inconsistent solutions, inheriting genes with items already transmitted to the solution. In that instance, the whole gene is removed, and the items are freed. To fix the solutions and make them consistent, the free items are reinserted with the FFD heuristic. This grouping crossover operator had not been used to solve the 1D-BPP, but it was used for the multiple knapsack problem [40].

4.2. Exon Shuffling Crossover

The next group-oriented crossover operator considered is ESX, which generates a single offspring C from two-parent solutions P_1 and P_2. ESX operates as follows. First, it merges the genes of the parents P_1 and P_2. Next, it shuffles them and later sorts them in descending order concerning their filling. Then, ESX performs the genetic material transmission following the order of the genes, and if any of the genes i has an element already inherited in another gene in the offspring solution, it is omitted. Finally, as in UX, ESX applies the FFD heuristic to reinsert the released items into the solution C, ensuring the feasibility and consistency of the offspring solution. ESX has shown outstanding results; therefore, the specialized literature includes different GGAs that use this crossover operator to address the 1D-BPP [6,41–43].

4.3. Greedy Partition Crossover

Another group-oriented crossover operator considered in this research is GPX. Similar to UX, GPX uses two parent solutions P_1 and P_2 to generate two children solutions C_1 and C_2; however, it relies on a greedy heuristic to guide the recombination process. The operator is described below. First, it sorts both parents in descending order concerning the fullness of their genes. Next, it generates a pair of vectors equal in size to the parent with the largest number of genes, which consists of uniformly distributed values in the interval (0,1), one vector for each offspring. Thus, GPX controls the genetic material transmission as follows. In the case of the first child C_1, if its corresponding vector contains a value less than or equal to 0.5 at position i, then it receives the gene i from the first parent P_1; otherwise, it receives the gene i from the second parent P_2. Similarly, for each position i in the vector corresponding to the second child C_2, if there is a value less than or equal to 0.5, then it receives the gene i from the second parent P_2; otherwise, it receives the gene i from the first parent P_1. During this process, if a gene contains items that have already been inherited by another gene in the solution, those elements are omitted to avoid duplication. As a result, some elements may remain unassigned after the recombination stage. Finally, the First Fit Decreasing (FFD) heuristic is applied to reinsert the remaining free items into each solution, ensuring viability and consistency. According to the specialized literature, this operator has been used to solve the 1D-BPP twice [6,44].

4.4. Gene-Level Crossover

GLX is the predefined operator in the GGA-CGT. Like UX and GPX, GLX generates two offspring solutions C_1 and C_2 from two-parent solutions P_1 and P_2. Its operational procedure is detailed below. First, GLX sorts the parents P_1 and P_2 in descending order concerning the filling of their genes. Subsequently, this operator performs the genetic material transmission by comparing the parents' genes in parallel one by one, prioritizing

Table 1. Main features of ESX, GPX, UX, GLX, and GLX-1 operators.

Operator	Number of children	Ordering of the genes	Probability	All genes
ESX	1	✓	×	✓
GPX	2	✓	✓	×
UX	2	×	✓	×
GLX	2	✓	×	✓
GLX-1	1	✓	×	✓

the transmission of the fullest gene. It is important to note that if both genes have the same filled, the first child C_1 receives the gene from the first parent P_1 and then the one from the second parent P_2. Similarly, the second child C_2 receives first the gene from the second parent P_2 and later the one from the first parent P_1. Alike UX, ESX, and GPX, this crossover operator also may generate inconsistent offspring, transmitting genes with repeated items. In such cases, GLX removes the genes with repeated genetic material and releases their items. Finally, the free items are reinserted into the solution using the FFD heuristic. According to the scope of the literature review, this operator has been used in different occasions to solve the 1D-BPP [4,45–47].

4.5. Gene-Level Crossover-1

The last group-oriented crossover operator included in this study is GLX-1, a variant of GLX that generates a single offspring C. The genes of the parents P_1 and P_2 are sorted in descending order concerning the filling of their genes. After that, the transmission begins, for which the parents' genes are compared one by one in parallel, prioritizing the transmission of the fullest gene. If both parents are equally filled, the child C receives the gene from the first parent P_1. Like GLX, this crossover operator may generate inconsistent solutions, transmitting genes with repeated items. In such cases, GLX-1 removes the genes with repeated genetic material and releases their items. Finally, the free items are reinserted into the solution using the FFD heuristic. Since this operator is one of the newest, it has only been used to solve de 1D-BPP once [6].

5. Computational Experiments

This section presents the experimental procedure conducted to evaluate different crossover operators within the GGA-CGT. The objective is to quantify their impact on the algorithm's performance and identify the characteristics that most influence it.

To this end, the experiment employed a combined benchmark composed of the classical test sets and the more recent BPPv_u_c benchmark introduced by Carmona-Arroyo et al. in 2021 [48], wich consists of four classes, called BPP.25, BPP.5, BPP.75, and BPP1. It has 700 instances per class, divided by seven capacities, each comprising one hundred instances. On the other hand, the classical benchmark comprises 1,615 instances distributed across nine classes [5,49–52]. In all cases, the optimal solutions are known. Table 2 summarizes the main characteristics of each class, including the number of instances (Inst.), bin capacity (c), the range of number of items (n), the distribution of item weights (w_j), the statistical distribution used to generate them, the presence of items heavier than half the bin capacity (Big items), and whether the optimal solution requires bins to be packed at full capacity (Perfect packing).

For this study, the original implementation of the GGA-CGT described in Section 3 [4] was used, with all parameter settings kept unchanged. The complete baseline parameter configuration is summarized in Table 3. The experimental procedure is structured into eight distinct phases as follows. Phase 1, State-of-the-Art Grouping Crossover Operators, ana-

Class	Inst.	c	n	w_j	Items distribution	Big items	Perfect packing
Dataset 1	720	100,120,150	[50,500]	[1,100]	Uniform		
Dataset 2	480	100	[50,500]	[100,300]	Uniform		
Dataset 3	10	100,000	200	[20000,35000]	Uniform	x	
Triplets	80	100	[60,501]	[25,50]	Ad-hoc		x
Uniform	80	150	[120,1000]	[20,100]	Uniform		
Hard 28	28	10,000	[160,200]	[1,800]	Ad-hoc	x	x
Was 1	100	1,000	100	[150,200]	Uniform		
Was 2	100	1,000	120	[150,200]	Uniform		
Gau 1	17	10,000	[57,239]	[2,7332]	Ad-hoc	x	x
BBP.25	700	$[10^2 - 10^8]$	[110,154]	(0,0.25c]	Ad-hoc	x	x
BPP.5	700	$[10^2 - 10^8]$	[124,167]	(0,0.50c]	Ad-hoc	x	x
BPP.75	700	$[10^2 - 10^8]$	[132,165]	(0,0.75c]	Ad-hoc	x	x
BPP1	700	$[10^2 - 10^8]$	[148,188]	(0,c]	Ad-hoc	x	x

Table 2. Main Characteristics of the instances in each class of the benchmark

lyzes the impact of five grouping crossover operators on the performance of the GGA-CGT. Phase 2 examines the influence of different gene ordering methods within the crossover process, while Phase 3 evaluates different gene transmission strategies from parent solutions to offspring. Building upon these findings, Phase 4 analyzes the impact of the crossover operator derived from the previous phases when executed under multiple configurations for the parameter n_c. Phase 5 assesses the overall performance of the crossover operator using different indicators of the algorithmic process. Based on the insights obtained, Phase 6 proposes a control method for the n_c parameter. Phase 7 incorporates a heuristic into the solution repair process during crossover. Finally, Phase 8 integrates control schemes for the mutation operator and its auxiliary parameters (k's), in order to evaluate their impact on performance, resulting in an adaptive and robust algorithm. The following sections describe each experimental phase in detail, present the corresponding results, and provide a discussion of the main observations and conclusions.

Table 3. Baseline parameter configuration of the GGA-CGT [4].

Parameter	Value
Population size $\|P\|$	100
Maximum number of generations max_gen	500
Number of individuals selected for crossover n_c	20
Number of individuals selected for mutation n_m	83
Rate of change for non-cloned solutions k_{ns}	1.3
Rate of change for cloned solutions k_{cs}	4
Maximum survival age of elite individuals $life_span$	10

5.1. Phase 1: State-of-the-Art Grouping Crossover Operators

As a first experimental step, an analysis was conducted to assess the impact of five crossover operators, ESX, GPX, UX, GLX, and GLX-1, on the performance of the GGA-CGT across all benchmark instances. For this study, the original implementation of the GGA-CGT described in Section 3 [4] was used, keeping all its parameter settings unchanged. To ensure a fair and controlled comparison, the baseline algorithm remained unchanged except for the crossover operator, which was replaced by each of the five variants under study. As a result, five GGA-CGT variants were obtained, denoted as GGA-CGT+ESX, GGA-CGT+GLX, GGA-CGT+GLX1, GGA-CGT+GPX, and GGA-CGT+UX. Each variant was evaluated across ten values n_c, ranging from n_c = 0 to n_c= 90, in increments of 10. This preliminary evaluation aimed to analyze the sensitivity of algorithmic performance

to the n_c parameter and to identify operator-specific properties that strongly influence the effectiveness of the evolutionary search.

Figure 2 shows the experimental results of this stage. The x-axis indicates the evaluated values for n_c, while the y-axis illustrates the number of optimal solutions discovered by GGA-CGT for each crossover operator, represented by distinct colors. Each bar corresponds to the number of optimal solutions attained by each GGA-CGT variant across the evaluated n_c values. As can be observed, GGA-CGT achieves the best performance with high n_c value (≥ 80) for the ESX, GPX, and GLX-1 operators. Interestingly, for UX and GPX, the best performance is achieved when no crossover is applied.

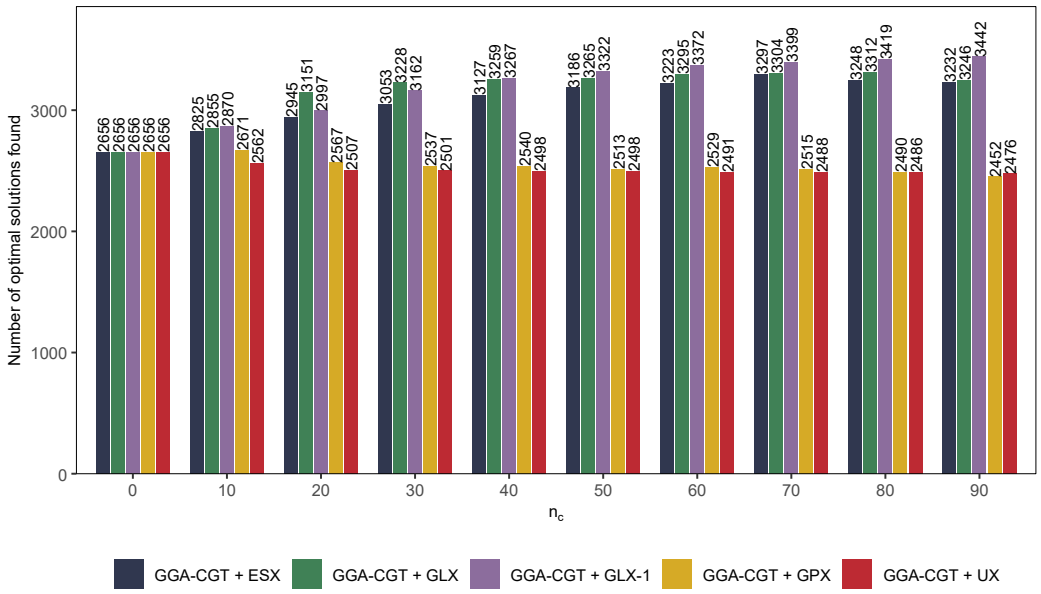


Figure 2. Number of optimal solutions obtained by the GGA-CGT with the operators: ESX, GPX, UX, GLX, and GLX-1 for the 1D-BPP.

Furthermore, the data shows that the algorithm achieving the highest number of optimal solutions is GGA-CGT+GLX-1, using $n_c = 90$ individuals for the crossover process, solving 3342 out of the 4415 instances of the combined benchmark. This configuration solves 196 more instances than the original GGA-CGT using the default GLX operator. This behavior can be associated with the reduced number of offspring generated by GLX-1, as it produces only one child per pair of parents, which may promote a more focused and stable transmission of genetic material. It is worth noting that, with a parameter value of $n_c = 90$ and two offspring generated per mating event, the GLX operator replaces almost the entire population in each generation. This high replacement pressure may lead to rapid loss of diversity, which could explain why GLX-1, producing only one offspring per pair of parents, achieves better performance under the same n_c value.

The preliminary evaluation revealed that the performance of the GGA-CGT is strongly influenced by (i) the characteristics of the problem instances, (ii) the crossover operator employed, and (iii) the configuration of the parameter n_c. These findings suggest that the design and integration of purpose-specific crossover strategies can have a direct and substantial impact on the overall effectiveness of the GGA-CGT. According to Ramos-Figueroa & Quiroz-Castellanos (2024) [7], the crossover procedure can be decomposed into three main stages: (1) the strategies to sort the genes in the parents, (2) the heuristics to perform the genetic material transmission, and (3) the heuristics to reinsert the items released during recombination. Hence, the following phases studies the influence of each stage independently. In each phase, different heuristic strategies are evaluated for

a single stage of the crossover process, while all other components are kept fixed. This controlled setting allows the isolated analysis of sorting strategies, transmission heuristics, and reinsertion mechanisms, enabling the quantification of their individual contributions to the quality of the generated offspring. To take advantage of the knowledge gained from previous experimentation, all experiments will be performed using the GLX-1 operator as the base template and the same state-of-the-art GGA-CGT parameter configuration.

5.2. Phase 2: Strategies to sort the genes in the parent solutions

As first stage of the crossover process, the performance of seven strategies, used to sort the genes (bins) of the parent solutions before the crossover takes place, was evalueted. These strategies are defined based on three criteria commonly used in packing heuristics: the bin filling level, the number of items contained in each bin, and the total weight of those items. The first three strategies order genes according to one of the following criteria: Random (R), Fullness (F), and Items (I). Specifically, R applies a uniform random permutation to the gene sequence, F arranges genes in descending order of fullness, and I ranks genes in ascending order based on the number of items. The remaining strategies rely on combinations of these criteria: Fullness–Items (FI), Fullness–Random (FR), Items–Fullness (IF), and Items–Random (IR). The FI strategy orders genes primarily by decreasing fullness, resolving ties using the number of items and prioritizing genes with fewer items. Similarly, FR prioritizes genes by decreasing fullness but resolves ties at random. The IF strategy ranks genes according to an increasing number of items, breaking ties based on bin fullness and favoring fuller bins. Finally, IR also prioritizes genes with fewer items, resolving ties randomly. As in phase 1, for this experiment, the state-of-the-art GGA-CGT and its parametrization [3] were used as the base template, combined with the best-performing operator from the first experiment, the GLX-1. This allows to isolate the effect of sorting strategies to quantify their individual contribution to the quality of the generated offspring. As a result, seven GGA-CGT variants were evaluated: GGA-CGT + GLX-1 + R, GGA-CGT + GLX-1 + F, GGA-CGT + GLX-1+I, GGA-CGT + GLX-1 + FI, GGA-CGT + GLX-1 + FR, GGA-CGT + GLX-1 + IF, and GGA-CGT + GLX-1 + IR. Furthermore, since this phase involves evaluating many strategies independently, the experiments were conducted on a stratified 20% subset of the benchmark, ensuring proportional representation across all classes. Specifically, 144 instances were chosen from Data Set 1, 96 from Data Set 2, 2 from Data Set 3, 16 from Triplets, 16 from Uniform, 6 from Hard28, 20 from Was 1, 20 from Was 2, and 4 from Gau 1. Additionally, 140 instances were selected from each of the BPP.25, BPP.5, BPP.75, and BPP1 categories. Table 4 summarizes the results obtained by the evaluated strategies. Each row reports the number of optimal solutions achieved per problem class. The first column identifies the problem class, the second column indicates the total number of instances in that class, and the remaining columns report the number of optimal solutions obtained by the GGA-CGT+GLX1 using each sort strategy.

As reported in Table 4, the GGA-CGT + GLX-1 + FI variant achieved the best performance. This configuration employs a sorting strategy that orders genes according to bin fullness, using the number of items as a tie-breaking criterion. It optimally solved 621 instances, corresponding to 70.33% of the total. In contrast, the GGA-CGT + GLX-1 + IR variant showed the weakest performance, optimally solving 531 instances (60.16%). This variant uses a sorting strategy based on the number of items, with ties broken at random. Consequently, in the next phase, the GGA-CGT + GLX-1 + FI configuration is adopted, keeping the gene-sorting strategy fixed.

Class	Cases	GGA-CGT + GLX-1						
		R	F	I	FI	FR	IF	IR
Data set 1	144	143	144	143	144	144	144	143
Data set 2	96	96	96	96	96	96	96	96
Data set 3	2	2	2	2	2	2	2	2
Triplets	16	16	16	16	16	16	16	16
Uniform	16	16	16	16	16	16	16	16
Hard 28	6	4	4	3	4	4	4	3
was 1	20	20	20	20	20	20	20	20
was 2	20	20	20	20	20	20	20	20
gau 1	3	3	3	3	3	3	3	3
BPP.25	140	75	83	78	84	83	79	78
BPP.5	140	58	64	51	74	65	59	50
BPP.75	140	40	48	41	52	48	42	40
BPP1	140	43	91	46	90	84	47	44
Total	883	536	607	535	**621**	601	548	531

Table 4. Number of instances optimally resolved by the GGA-CGT+ GLX-1 with the strategies to sort genes: R, F, I, FI, FR, IF, and IR.

5.3. Phase 3: Strategies for Genetic Material Transmission

Once the gene-ordering strategies had been analyzed, the experimental study focused on evaluating alternative gene transmission mechanisms. This phase aimed to define a general transmission scheme based on the parallel comparison of the ordered genes of both parents, considering one gene position at a time. Under this scheme, each offspring may inherit genetic material from either or both parents, depending on the selected transmission criterion.

The strategies examined in this phase follow the same criteria used to define the gene-ordering mechanisms, R, F, I, FI, FR, IF, and IR, adapted to the gene transmission process. In all cases, the genes of both parents are processed in parallel, and the strategies differ only in the rule used to decide which of the two competing genes is transmitted first. Under this scheme, strategy R selects the gene to be transmitted first using a uniformly distributed random value. On the other hand, strategies F and I prioritize the fullest gene and the gene containing the fewest items, respectively. In both cases, ties are resolved by selecting the gene from the first parent. In the case of combined strategies, a primary selection criterion is applied, while the secondary criterion is used for tie-breaking. In this way, strategy FI prioritizes the fullest gene and resolves ties by favoring the gene with fewer items, while strategy, FR also prioritizes the fullest gene but resolves ties randomly using a uniform distribution. In contrast, strategy IF transmits first the gene with fewer items and breaks ties based on fullness, favoring the fullest bin; while strategy IR also prioritizes genes with fewer items and resolves ties randomly using a uniform distribution.

As in Phase 2, the GGA-CGT variant GGA-CGT+GLX-1+FI was used as the reference configuration in this study in order to isolate the effect of the gene transmission strategies and quantify their individual contribution to offspring quality. It is important to note that, prior to selecting this configuration, all possible combinations between the gene-ordering strategies evaluated in Phase 2 and the gene transmission strategies analyzed in this phase were experimentally assessed. This exhaustive evaluation ensured that potentially favorable interactions between different ordering and transmission criteria were not excluded. However, for clarity and conciseness, only the results corresponding to the best-performing combinations are reported. These combinations consistently involved the FI gene-ordering strategy, which demonstrated superior and more robust performance

Class	Cases	GGA-CGT + GLX-1 + FI						
		R	F	I	FI	FR	IF	IR
Data set 1	144	144	144	144	144	144	143	143
Data set 2	96	96	96	96	96	96	96	96
Data set 3	2	2	2	2	2	2	2	2
Triplets	16	15	16	16	16	16	16	16
Uniform	16	16	16	16	16	16	16	16
Hard 28	6	4	5	4	4	4	3	3
was 1	20	20	20	20	20	20	20	20
was 2	20	20	20	20	20	20	20	20
gau 1	3	3	3	3	3	3	3	3
BPP.25	140	92	89	88	91	88	67	63
BPP.5	140	77	73	77	76	77	48	44
BPP.75	140	58	55	51	56	55	40	40
BPP1	140	90	82	84	86	85	41	42
Total	883	**637**	621	621	630	626	515	508

Table 5. Number of instances optimally resolved by the GGA-CGT+ GLX-1 + FI with the strategies to transmit the genes: R, F, I, FI, FR, IF, and IR.

across all evaluated instances. As a result, seven GGA-CGT variants were evaluated: GGA-CGT + GLX-1 + FI/R, GGA-CGT + GLX-1 + FI/F, GGA-CGT + GLX-1 + FI/I, GGA-CGT + GLX-1 + FI/FI, GGA-CGT + GLX-1 + FI/FR, GGA-CGT + GLX-1 + FI/IF, and GGA-CGT + GLX-1 + FI/IR.

Table 5 summarizes the results obtained by applying the seven gene transmission strategies within the GLX-1 operator. As in Phase 2, each row reports the number of optimal solutions achieved for each problem class. The first column identifies the problem class, the second indicates the total number of instances per class, and the remaining columns present the number of optimal solutions obtained by each GGA-CGT + GLX-1 + FI configuration under the different gene transmission strategies.

As reported in Table 5, the GGA-CGT + GLX-1 + FI/R variant achieved the highest performance. This configuration employs the R genetic material transmission strategy, which introduces stochasticity to determine the order in which genes are transmitted. This approach yielded a total of 637 optimal solutions, which is 16 more than the default algorithm (Fullness), representing 72.1% of the total test cases. This transmission strategy demonstrated better performance, in the BPP.25, BPP.5, BPP.75, and BPP1 classes. Conversely, for the classes Hard28 and Triplets, other strategies showed superior performance in terms of the number of optimal solutions. In contrast, the IR strategy performed the least favorably, solving only 508 optimal instances, 113 fewer than the original version.

5.4. Phase 4: Parameter setting of the n_c parameter

After identifying FI and R as the most suitable heuristics for parental gene ordering and genetic material transmission, respectively, within the GLX-1 operator, the study proceeded to evaluate the full benchmark. In this phase, the parameter n_c was varied from 10% to 100%, while the remaining parameters were kept unchanged and correspond to the baseline configuration reported in Table 3.

This analysis aims to assess the impact of n_c, defined as the number of individuals selected for crossover in each generation, on the algorithmic performance, as well as to compare the resulting configurations against the original GGA-CGT.

Table 6 summarizes the experimental results obtained by the GGA-CGT+GLX-1+FI/R for each value of n_c. The first two columns report the problem class and the total number

Class	Cases	n_c									
		10%		20%		30%		40%		50%	
		Opt.	Gen.	Opt.	Gen.	Opt.	Gen.	Opt.	Gen.	Opt.	Gen.
data set 1	720	720	131.29	719	131.95	720	131.16	719	131.86	718	132.41
data set 2	480	480	24.61	480	24.67	480	24.65	480	24.68	480	24.71
data set 3	10	10	363.60	10	360.10	10	375.60	10	360.10	10	367.20
triplets	80	79	38.45	79	44.54	80	43.91	80	43.75	80	44.33
uniform	80	80	11.00	80	10.11	80	11.66	80	11.71	80	11.38
hard28	28	14	385.86	15	371.57	15	376.89	15	415.71	13	397.82
was1	100	100	0.84	100	0.78	100	0.62	100	0.68	100	0.62
was 2	100	100	187.13	100	185.42	100	185.64	100	185.28	100	185.26
gau 1	17	16	110.00	16	105.88	16	104.29	16	102.35	16	102.41
BPP.25	700	400	232.22	445	224.59	492	206.43	500	192.54	500	188.47
BPP.5	700	326	294.47	367	275.79	384	264.51	387	259.26	389	253.97
BPP.75	700	234	347.98	273	332.21	307	314.84	353	294.67	374	283.53
BPPP1	700	345	301.64	449	246.49	515	204.36	557	170.04	579	150.14
Total	4415	2904	186.85	3133	178.01	3299	172.66	3397	168.66	3439	164.79

Class	Cases	n_c									
		60%		70%		80%		90%		100%	
		Opt.	Gen.	Opt.	Gen.	Opt.	Gen.	Opt.	Gen.	Opt.	Gen.
data set 1	720	718	132.43	719	131.93	719	132.20	718	132.38	719	131.87
data set 2	480	480	24.72	480	24.68	480	24.92	480	24.71	480	25.12
data set 3	10	9	402.40	10	361.30	9	402.40	9	402.70	9	403.40
triplets	80	78	66.53	80	59.15	79	66.94	78	80.79	80	68.05
uniform	80	79	15.11	79	15.19	79	15.15	79	15.19	79	15.73
hard28	28	12	419.54	12	417.07	13	419.57	12	440.54	10	462.86
was1	100	100	0.61	100	0.58	100	0.57	100	0.57	100	0.53
was 2	100	100	185.24	100	185.58	100	185.29	100	185.82	100	185.46
gau 1	17	16	98.35	16	97.94	16	98.06	16	99.06	16	98.71
BPP.25	700	502	182.95	501	181.65	500	177.77	502	175.62	505	176.27
BPP.5	700	393	250.71	391	249.12	390	248.49	387	248.61	390	247.50
BPP.75	700	419	261.48	444	249.40	474	223.43	472	223.32	476	219.44
BPPP1	700	578	141.08	599	126.47	598	122.06	613	111.56	610	103.28
Total	4415	3484	167.78	3531	161.54	3557	162.83	3566	164.68	**3574**	164.48

Table 6. Optimal solutions obtained by the GGA-CGT with varying percentages of crosses, ranging from 10% to 100%, as well as the number of generations required to achieve these results.

of instances evaluated per class. For each n_c configuration, results are presented in pairs of columns: the first column (Opt.) indicates the number of instances for which an optimal solution was achieved, while the second column (Gen.) reports the average number of generations required to reach such solutions within that class.

The bottom row of each table section aggregates the results across all instance classes, reporting the total number of optimal solutions obtained and the overall average number of generations. For presentation purposes, the results are displayed in two complementary table segments, each covering a subset of the evaluated n_c values, in order to improve readability without affecting the interpretation of the results.

From Table 6 it can be observed that the highest number of optimal solutions is achieved when using a $n_c = 100$, yielding a total of 441 additional optimal solutions compared to the original configuration (20%). Regarding classes, for the data set 2, triplets, uniform, was 1, was 2, and gau 1, no considerable sensitivity is observed with respect to the percentage of crossover, as the number of optimal solutions remains largely stable. Conversely, the classes data set 1 and data set 3 show some variations, although they are not substantial and may be more attributable to randomness. Interestingly, the class Hard28 exhibits a distinct behavior, achieving a higher number of optimal solutions with relatively

low values for the n_c parameter (below 50%). Although the variation ranges from 1 to 5 optima, this difference is significant considering that the class comprises only 28 instances and has been consistently identified as highly challenging. Finally, for the BPP.25–BPP1 classes, an increasing trend is observed as the n_c value rises, with most cases reaching the maximum number of optimal solutions at 100% crossover. The exception is the BPP.5 class, which attains its best performance at n_c=60.

Given these observations, a detailed analysis to understand the algorithmic behavior of the crossover operator was proposed for the next phase, conducted with instances belonging to classes that exhibited distinctive performance patterns.

5.5. Phase 5: Analysis of the GGA-CGT Algorithmic behavior with the GLX-1 Operator and the FI/R Heuristic Strategies

To gain a deeper understanding of how the GLX-1 operator combined with the FI/R heuristic strategies influences the algorithmic behavior of the GGA-CGT, an additional experiment was designed. In this phase, seven representative instances were selected, each corresponding to a distinct class from data set 1, triplets, hard28, BPP.25, BPP.5, BPP.75, and BPP1. For every instance, four values of the parameter n_c were considered: 0, 20, 60, and 100, corresponding to the extreme settings, the original configuration, and an intermediate value. In this way, each instance was solved four times. To present a complete overview, a pool of indexed were defined to obtain information from three different aspects. First, to examine the overall GGA-HIS+GLX-1+FI/R algorithmic performance, the following metrics were collected: (1) the individual with the highest fitness (*Best*), (2) the individual with the lowest fitness (*Worst*), (3) the mean fitness of the population (*Average*), and (4) the average number of individuals with repeated fitness values (*Repeated*). On the other hand, to analyze the crossover process and to gain insight into how solutions are recombined and structured, three indicators were used, covering (1) the mean number of fully filled bins in the population after crossover (*Average ratio*), (2) the proportion of items freed during crossover (*Free items*), and (3) the proportion of bins filled to their maximum capacity in the resulting solutions (*Partial bins*). Finally, to evaluate the global behavior and internal structure of the best and worst overall solutions, two measures were defined. These measures analyze the proportion of bins filled to their maximum capacity in the best and worst population solutions after the crossover process and are referred to as *Best crossover ratio* and *Worst crossover ratio*, respectively.

Figures 3–5 provide a graphical summary of the collected data. Each figure organizes the plots into seven rows, one for each instance under study, and four plots per row corresponding to the evaluated n_c values. Each row of plots also includes descriptive information about the instance, including its name, number of items, bin capacity, and the number of bins in the optimal solution. In all plots, the x-axis represents the generation number, while the y-axis reports the indicators analyzed.

Figure 3 shows the overall GGA-HIS + GLX-1+FI/R algorithmic performance, with the indicators *Best, Worst, Average*, and *Repeated*, represented in dark blue, red, and green lines, and yellow marks, respectively. These results suggest that, in most instances, the fitness of the best individual stabilizes early, causing premature convergence, particularly at n_c=60 and n_c=100. With 0% crossover, the gap between the best and average fitness is larger, reflecting a more limited search; yet population diversity remains higher, as indicated by fewer repeated-fitness individuals across generations. Conversely, at 60% and 100%, convergence is faster and more pronounced, implying greater exploitation of the search space. However, the proportion of identical individuals increases over time, especially at 100% crossover, revealing a loss of diversity in later generations. This indicates a delicate balance: intense crossover accelerates convergence but may lead to premature stagnation.

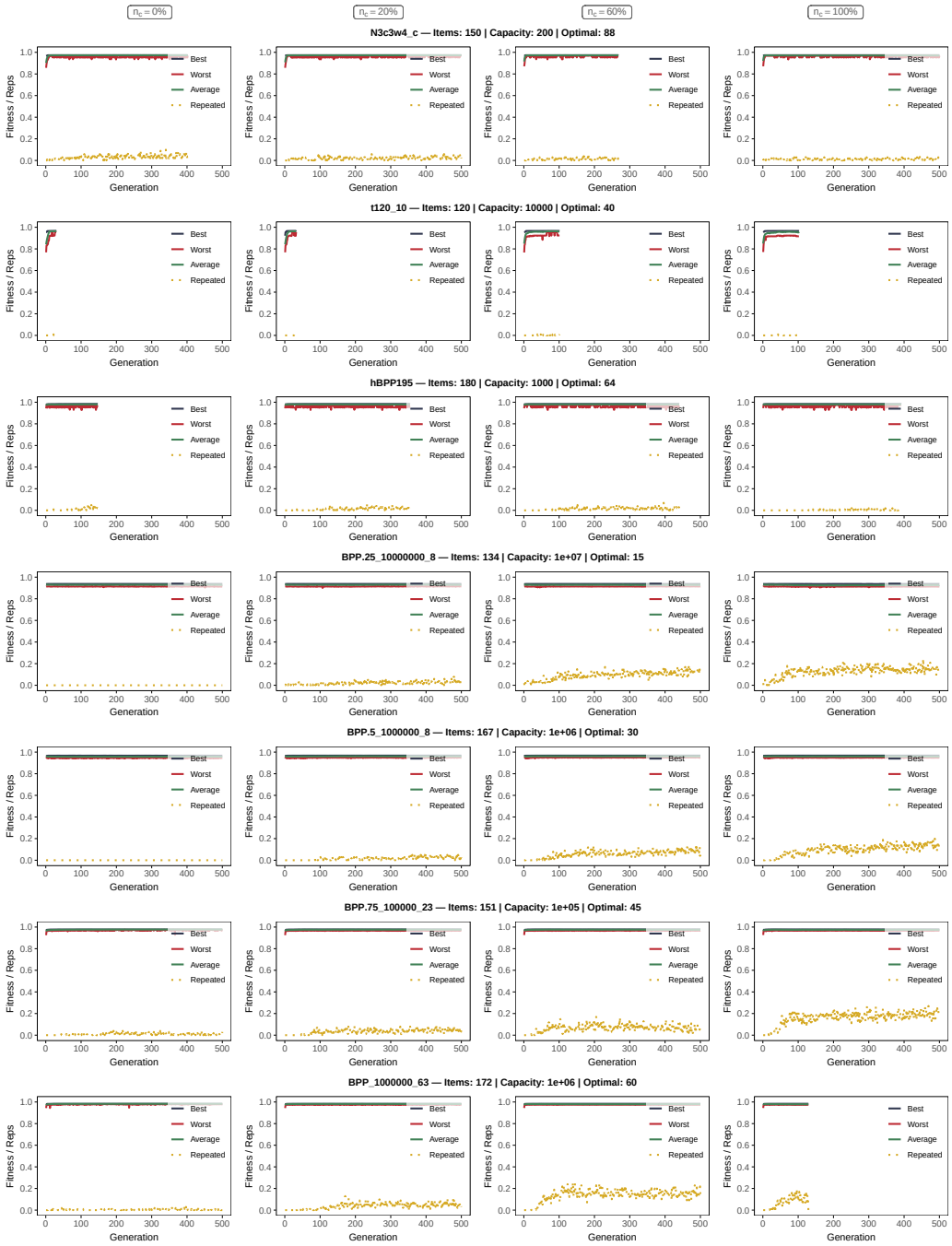


Figure 3. Overall GGA-HIS + GLX-1+FI/R algorithmic performance for n_c=0, 20, 60, and 100, using the indexes: *Best*, *Worst*, *Average*, *and Repeated*.

Zooming in to the instance level, t120_10 (triplets) and BPP.25_10000000_8 (BPP.25) exhibit higher variability across configurations, indicating greater sensitivity to the problem structure. In both, the bin capacities are very large relative to the number and size of the items, resulting in broad and irregular search spaces where small genetic changes can have a significant impact on fitness. In contrast, N3c3w4_c (data set 1) and BPP.75_100000_23 (BPP.75) show smoother and more stable convergence trajectories. In the latter, although the capacity is high, items can occupy up to 75%, limiting the number of items per bin and promoting more compact and regular configurations. Consequently, the proposed

crossover operator preserves structural coherence in the packings, demonstrating greater stability and robustness without introducing pronounced fitness oscillations.

Regarding overall convergence, in most instances the fitness of the best individual stabilizes early, suggesting premature convergence, particularly at n_c values of 60% and 100%. With 0% crossover, the gap between the best and average fitness is larger, reflecting a more limited search; yet population diversity remains higher, as indicated by fewer repeated-fitness individuals across generations. Conversely, at 60% and 100%, convergence is faster and more pronounced, implying greater exploration and exploitation of the search space. However, the proportion of identical individuals increases over time, especially at 100% crossover, revealing a loss of diversity in later generations. This indicates a delicate balance: intense crossover accelerates convergence but may lead to premature stagnation.

At the instance level, t120_10 (triplets) and BPP.25_10000000_8 (BPP.25) exhibit higher variability across configurations, indicating greater sensitivity to the problem structure. In both, the bin capacities are very large relative to the number and size of the items, resulting in broad and irregular search spaces where small genetic changes can have a significant impact on fitness. In contrast, N3c3w4_c (data set 1) and BPP.75_100000_23 (BPP.75) show smoother and more stable convergence trajectories. In the latter, although the capacity is high, items can occupy up to 75% of it, limiting the number of items per bin and promoting more compact and regular configurations. Consequently, the proposed crossover operator preserves structural coherence in the packings, demonstrating greater stability and robustness without introducing pronounced fitness oscillations.

On the other hand, Figure 4 provides insight into the internal dynamics of the crossover process, extracting information from the recombination process and the structure of the offspring generated, using indicators like *Average ratio, Free items*, and *Partial bins*, depicted by blue, pink, and green lines, respectively. The results reveal an inverse trend between both variables: as the proportion of free items decreases, the proportion of filled bins increases, indicating an effective redistribution of items during crossover. For the 0% crossover configuration, the Partial bins and Free items lines remain constantly at zero throughout all generations, since no recombination process takes place between solutions. Nevertheless, the Average ratio curve reveals the population's internal packing dynamics. For instances N3c3w4_c, t120_10 and hBPP195, this value increases rapidly and tends to converge toward an optimal configuration, although slight stagnation can be observed in some cases. In contrast, BPP.25_10000000_8 and BPP.5_1000000_8 maintain values close to zero, suggesting limited exploration of the search space. Finally, BPP.75_100000_23 and BPP_1000000_63 display similar behavior, converging toward sub-optimal solutions, indicating that the algorithm stabilizes prematurely. At 60% and 100% crossover, the Partial bins and Average ratio curves reach higher and more stable values, indicating that the proposed operator promotes more compact and efficient packings by reducing the number of partially filled bins. However, the rapid decrease in free items toward the final generations reveals a progressive loss of structural diversity, consistent with the accelerated convergence observed in Figure 4. Across instances, t120_10 and BPP.25_10000000_8 show larger fluctuations, reinforcing their sensitivity to problem structure. In contrast, N3c3w4_c and BPP.75_100000_23 exhibit smoother and more consistent trends, indicating the robust and coherent behavior of the proposed operator in configurations with a larger number of items or more regular weight distributions.

Finally, Figure 5 provides insight into the global behavior and internal structure of the best and worst overall by means of the *Best crossover ratio* and *Worst crossover ratio* measures. As illustrated in Figure 5 distinct patterns emerge in this experiment. For the 0% crossover case, i.e., when no individual in the population undergoes the crossover operator, the proportions corresponding to the best and worst solutions reach a constant value and

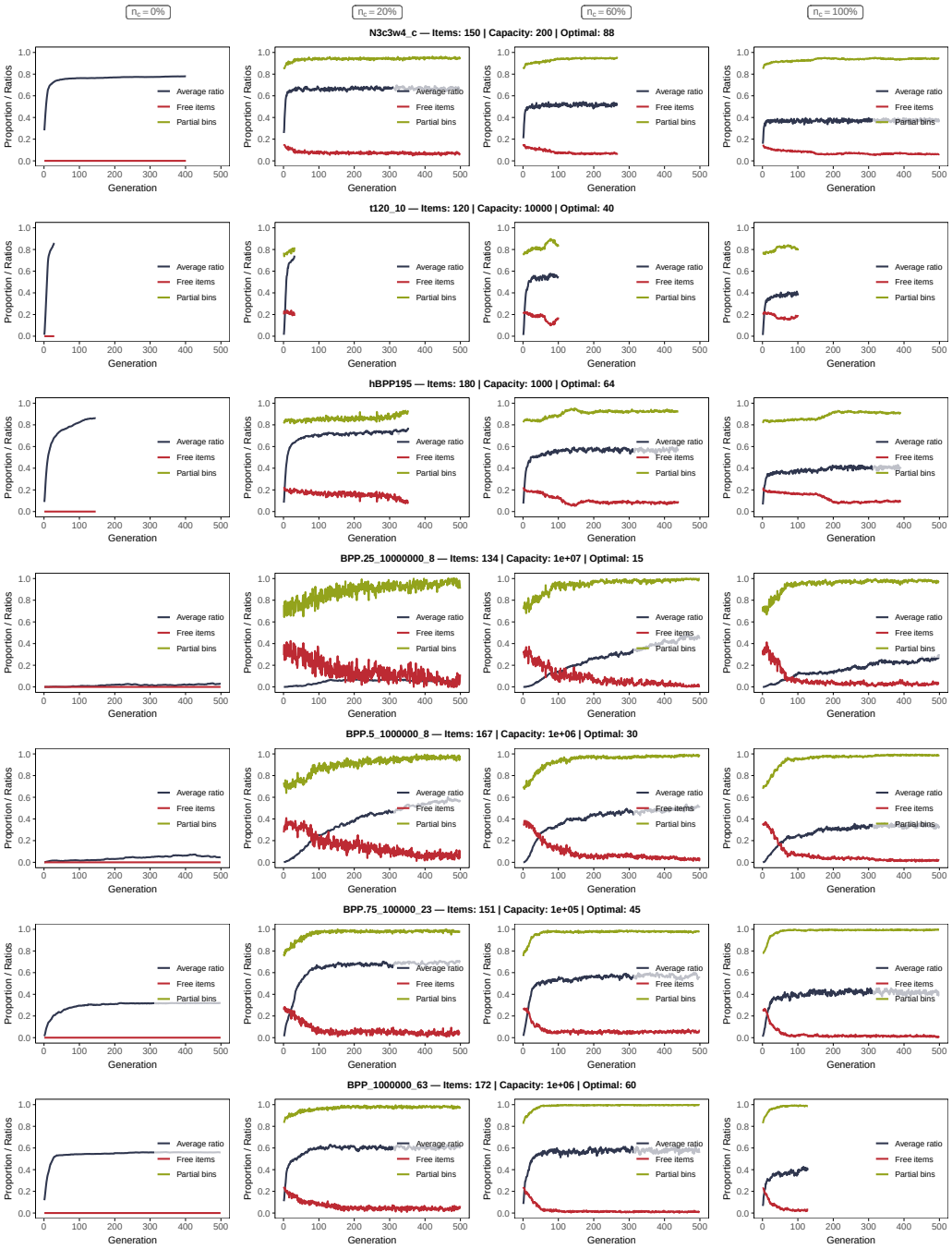


Figure 4. In-depth analysis of the recombination process of the GGA-CGT+GLX-1+FI/R and the structure of the offspring generated for n_c=0, 20, 60, and 100, using the indexes:*Average ratio, Free items, and Partial bins.*

remain equal throughout all generations. This behavior results from the complete absence of genetic information exchange, which keeps the population structurally stable and prevents significant improvements. It is worth noting that the best and worst solutions are not necessarily produced by the crossover operator, but rather by the general evolutionary process. Specifically, in instances N3c3w4_c, t120_10, and hBPP195, the growth in the proportion of fully filled bins is faster, and the optimal configuration is reached, whereas in the other four instances this does not occur. For BPP.25_10000000<_8 and BPP.5_1000000_8, the maximum observed proportion barely reaches a value close to 0.2, even though in

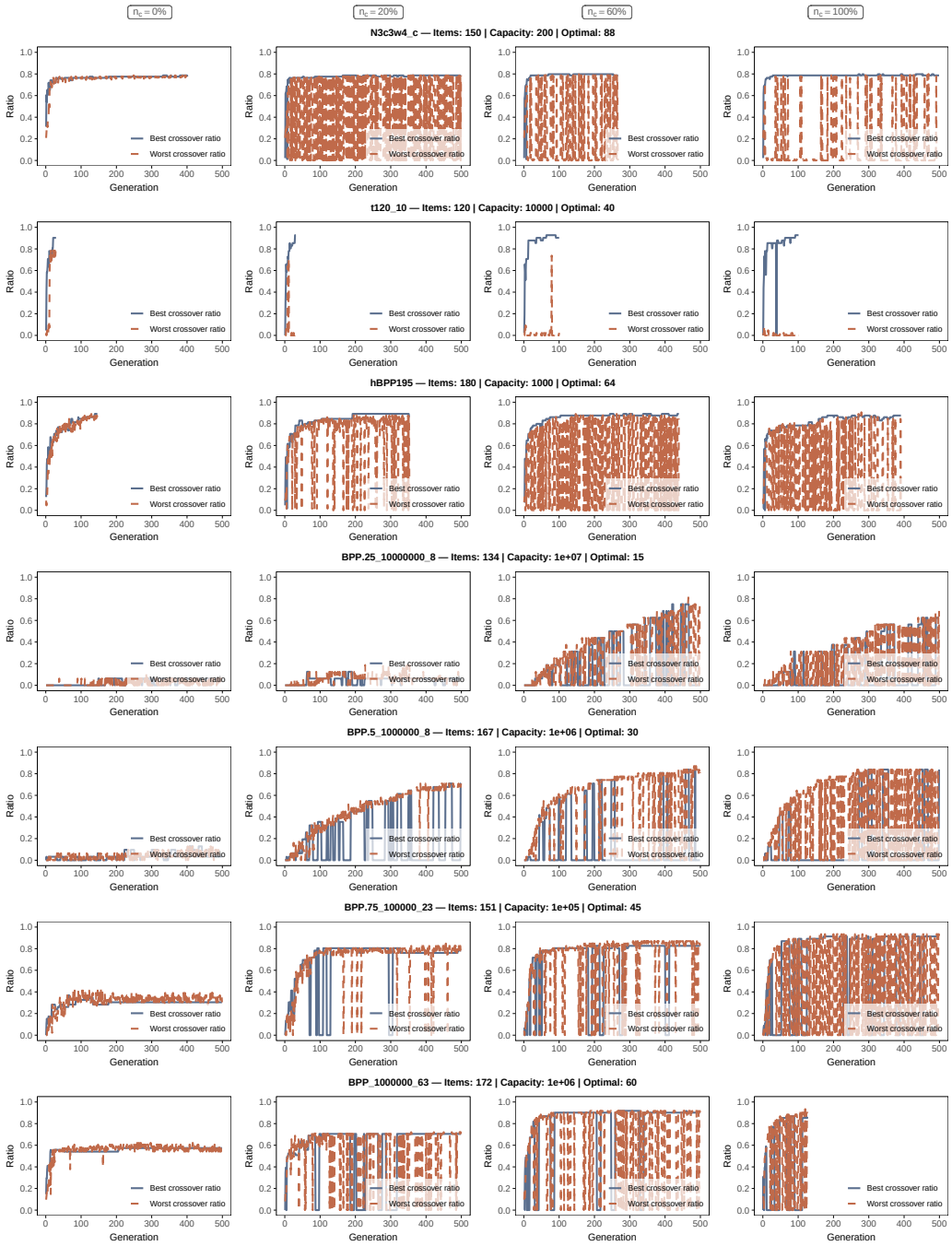


Figure 5. Global behavior and internal structure of the best and worst overall solutions for n_c=0, 20, 60, and 100, using the indexes: *Best crossover ratio and Worst crossover ratio*.

their optimal solutions all bins should be filled. With 20% crossover, the behavior is more moderate, showing gradual and stable growth in almost all cases. This suggests a balance between exploration and exploitation, where progress toward more efficient configurations occurs progressively, avoiding an abrupt loss of diversity. However, a particular phenomenon can be observed in the best solutions: none of them may contain bins filled, yet they can still achieve a higher fitness value than the rest of the population. This occurs mainly due to how fitness is computed, as it evaluates the overall efficiency in capacity utilization rather than the exact number of fully filled bins. Thus, a solution with bins that are almost filled in a homogeneous way may obtain a higher fitness value

than another with some bins completely full but others significantly underutilized. This behavior appears more frequently in BPP-type instances, where the distribution of weights and capacities produces multiple solutions that are very close in quality.

In contrast, with 60% and 100% n_c values, the proportion of fully filled bins in the best solution increases rapidly, reaching values close to 1, which indicates greater efficiency in space utilization and a faster approach to optimal configurations. In these same cases, the curve corresponding to the worst solution also shows an upward trend, although with greater variability, reflecting that the crossover operator induces overall improvement across the population, not only in the best individuals. Nevertheless, a similar pattern to the one previously described can still occur in some cases, the best solution may have no bins filled, except for instances N3c3w4_c and hBPP195, where this is achieved. Note that, in BPP.25_10000000_8 and BPP.5_1000000_8, a longer persistence of generations with best solutions lacking fully filled bins is observed, or frequent oscillations between configurations with none and others with some bins filled, despite the overall upward tendency. At the instance level, t120_10 and BPP.25_10000000_8 exhibit larger oscillations between the curves of the best and worst individuals, evidencing a more irregular search structure that depends strongly on problem characteristics. Conversely, N3c3w4_c, BPP.75_100000_23, and BPP_1000000_63 show smoother and more convergent trajectories, indicating that the proposed operator maintains greater consistency in the crossover process, even in more complex configurations.

Overall, the results obtained from the experimentation and the analysis of the three processes reveal a direct relationship between crossover intensity and the structural dynamics of the population. The variations in performance, proportions, and post-crossover behavior show that the proposed operator can promote more efficient and compact configurations in certain instances, although an excessive value of n_c may reduce diversity and lead to premature convergence in others. These findings highlight the importance of dynamically regulating crossover intensity, adapting it to the evolutionary state of the population to maintain an appropriate balance between exploration and exploitation according to the characteristics of each test case. Based on the results obtained, it becomes evident that the internal design choices of the crossover operator, particularly the ordering of parental genes and the mechanisms used to transmit genetic material significantly influence the algorithm's ability to maintain diversity and generate high quality offspring. However, these components interact directly with the parameter n_c itself, which determines how frequently these mechanisms are applied throughout the evolutionary process. These observations suggest that relying on a fixed n_c value may be insufficient, as the algorithm requires different levels of recombination pressure at different stages of the search. Consequently, the following section introduces a crossover control strategy designed to adaptively adjust this parameter during execution, with the goal of improving both stability and overall effectiveness.

5.6. Phase 6: An Adaptive Control Scheme for the n_c Parameter

The results obtained in Phase 5 suggest that employing a fixed n_c parameter constrains the algorithm's capacity to adapt its behavior to the dynamic conditions inherent in the evolutionary process. During the early generations, a high n_c value can promote exploration, enabling the algorithm to generate diverse offspring and escape poor regions of the search space. However, as the population begins to converge, applying a high n_c value may accelerate the loss of diversity and increase the risk of premature convergence. Conversely, selecting a very low n_c value during the initial stages restricts the formation of new genetic combinations, reducing exploration and hindering the discovery of promising solutions.

These findings highlight the need for a dynamic control scheme capable of adjusting the n_c value in response to the population's current state. The goal is to allow the algorithm to self-regulate the intensity of recombination by boosting exploration when diversity decreases and reinforcing exploitation when the population becomes more stable. In the previous experiments, a distinctive behavior was observed regarding the proportion of bins filled to their maximum capacity within the solutions: this value tends to oscillate between levels close to one and, at certain points in the process, may drop to zero. This observation inspired the idea of using this feature as a control measure for the n_c value, increasing it when the proportion of fully filled bins is low and decreasing it when this proportion rises. In this way, the crossover operator is regulated according to the structural quality of the solutions present in the population, promoting stronger exploitation in promising regions while maintaining a controlled balance between exploration and exploitation throughout the evolutionary process. Equation (10) presents the rule defined to control the n_c values.

$$n_c = \frac{1}{g} \sum_{i=1}^{g} r_i^{1/kcs}, \tag{10}$$

where n_c denotes the number of individuals to cross, g is the current generation number, r_i represents the proportion of incomplete bins in the best solution of the population immediately before the crossover process, and k_{cs} corresponds to the rate of change for cloned solutions. In this formulation, when r_i values are close to 1, meaning that very few bins are completely filled, the value of the parameter n_c increases, enhancing the exploration capability of the algorithm. Conversely, when r_i approaches 0, indicating that most bins are fully packed, the value of n_c decreases, reducing exploration and reinforcing exploitation in promising regions already identified by the population.

To evaluate the suitability of the proposed adaptive parameter control scheme, the GGA-CGT+GLX-1+FI/R algorithm designed in the previous phases was applied to the complete benchmark, which comprises 4415 instances. All algorithm parameters follow the same state-of-the-art baseline configuration reported in Table 3, with the sole exception of the parameter n_c, which is dynamically controlled according to Equation 10.

Table 7 summarizes the results obtained by the GGA-CGT+GLX-1+FI/R approach using the proposed adaptive parameter control scheme for n_c. The first column reports the instance class, followed by the number of cases per class, the number of optima achieved, and the number of generations required. Lastly, the final four columns present descriptive statistics of the n_c values used for each instance class, including the average (mean), the minimum (min), the maximum (max), and the standard deviation (sd). In this table, n_c is reported as a normalized proportion in the range $[0, 1]$, representing the fraction of the population selected for crossover (i.e., values can be directly interpreted as percentages in the range $[0, 100]\%$).

The results in Table 7 show a significant increase in the number of optimal solutions obtained compared to the algorithm with a fixed n_c parameter value, achieving 441 additional optimal solutions overall. Although the configuration with 100% crossover produced four more optimal solutions (see Table 6) than the adaptive control method, the proposed approach eliminates the need for manual parameter tuning, which represents a notable advantage in terms of robustness and algorithmic autonomy. This table also reveals that the global average value of n_c was 87, with values ranging between 58 and 100, indicating that the operator maintained relatively high recombination levels throughout the evolutionary process, ensuring consistent exploration. Certain instance classes, particularly those of the BPP type, benefited the most from the adaptive control, achieving higher efficiency and compact solution structures as the parameter n_c dynamically adjusted to the state of the population. Conversely, other classes such as hard28, data set 3, and triplets performed bet-

Class	Cases	Opt.	Gen.	n_c			
				Mean	min	max	sd
data set 1	720	718	132.41	0.85	0.00	1.00	0.85
data set 2	480	480	24.68	0.95	0.38	1.00	0.95
data set 3	10	8	450.30	0.93	0.86	1.00	0.93
triplets	80	79	56.15	0.97	0.92	1.00	0.97
uniform	80	80	11.88	0.78	0.56	0.98	0.78
hard28	28	7	473.75	0.71	0.38	1.00	0.71
was1	100	100	0.48	0.82	0.52	0.96	0.82
was 2	100	100	185.55	0.98	0.90	1.00	0.98
gau 1	17	16	100.06	0.92	0.78	1.00	0.92
BPP.25	700	504	176.67	0.92	0.66	1.00	0.92
BPP.5	700	393	244.80	0.93	0.56	1.00	0.93
BPP.75	700	472	223.16	0.81	0.56	1.00	0.81
BPPP1	700	613	113.25	0.77	0.50	1.00	0.77
Total	4415	3570	168.70	0.87	0.58	1.00	0.87

Table 7. Overall results of the GGA-CGT+GLX-1+FI/R with the adaptive parameter control scheme for n_c.

ter under lower and more stable crossover values, suggesting that excessive recombination can disrupt structural coherence in problems with more structurally sensitive or irregular solution spaces. In contrast, classes like was 2, triplets, and data set 2 reached higher average n_c (95–98) with full or near-full optimality, showing that the adaptive function correctly increased crossover intensity when additional exploration was beneficial. Overall, these results confirm that the proposed adaptive control method effectively regulates the crossover intensity according to the population's evolutionary dynamics and the structural characteristics of each instance class, enhancing both performance and autonomy without requiring additional parameter calibration.

Additionally, a similar approach to the one presented in Figures 3–5 was used to analyze the information of the evolutionary process presented by the GGA-CGT+GLX-1+FI/R using the nine indexes previously described: *Best*, *Worst*, *Average*, *Repeated*, *Average ratio*, *Free items*, *Partial bins*, *Best crossover ratio*, and *Worst crossover ratio*.

For this study, the same instances and the same color for every index were used, however, the overall layout of the plots was modified. Specifically, the information previously distributed across three separate figures is now consolidated into a single one. Figure 6 presents the information collected, each row of plots represents a different instance with 3 graphs, one with the indices to analyze the overall performance of the GGA-CGT+GLX-1+FI/R, another to examine the proportion of bin filling in the solutions generated by the crossover operator, and the last to analyze the n_c values of the best and worst solution plus a black line that indicates the values generated by the adaptive parameter control scheme proposed.

To understand the impact of the proposed control scheme, the analysis focuses mainly on the plots in the last column of Figure 6. Overall, the results reveal a progressive increase in the average proportion of fully filled bins across generations, along with a reduction in the number of free items, indicating a steady structural improvement in the solutions generated by the adaptive operator. Moreover, the parameter n_c behaves consistently with the evolutionary state of the population: when the proportion of fully filled bins decreases, indicating lower quality or diversity, the value of n_c increases, enhancing exploration. Conversely, when the proportion of filled bins rises, n_c gradually decreases, favoring exploitation of the most promising regions of the search space. Altogether, this pattern

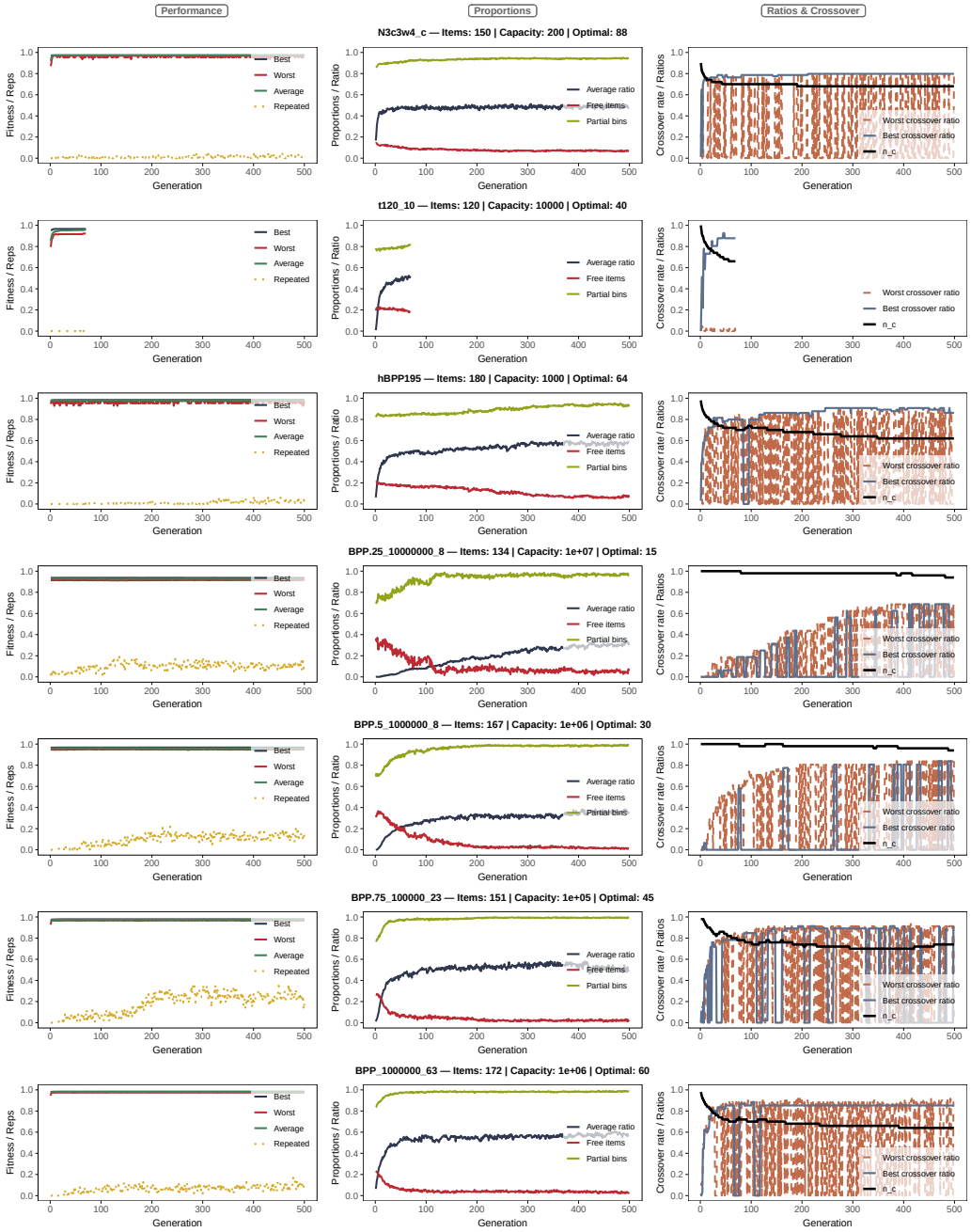


Figure 6. Performance of the GGA-CGT+GLX-1+FI/R with the proposed adaptive parameter control scheme.

confirms that the proposed control function successfully achieves its goal of adapting the crossover intensity dynamically, maintaining a balanced trade-off between exploration and exploitation throughout the evolutionary process.

5.7. Phase 7: The Repair Operator RP+EXT – An improvement to the GLX-1+FI/R operator

As mentioned earlier, the crossover operator may generate inconsistent solutions with missing items. This occurs because some items can be released during the genetic transmission process. To address this issue, the default strategy in the GLX-1+FI/R of the GGA-CGT is FFD. However, this approach is not always the most suitable, as in several scenarios many generated solutions were observed to not contain a single bin filled to its

maximum capacity. In such cases, it becomes convenient to incorporate a mechanism that improves the filling level after crossover. For this purpose, the Rearrangement by Pairs (RP) operator, introduced in [8], was employed, a new version where the process starts by sorting the free items produced after genetic transmission in ascending order according to their weight. Once sorted, a controlled exchange procedure is applied with the goal of improving bin utilization without violating capacity constraints. This enhanced version of the operator, referred to as RP-EXP, introduces a more structured reorganization process that allows for a finer adjustment of solution quality after genetic transmission.

The procedure evaluates three types of exchanges in a predefined order. First, a single free item is considered as a replacement for two packed items, provided that the resulting bin filling is maintained or improved without exceeding capacity. If no improvement is obtained, pairs of free items are assessed as substitutes for pairs of packed items under the same feasibility conditions. Finally, when neither of the previous cases is feasible, one-to-one exchanges are attempted, again ensuring that bin capacity constraints are satisfied and that the filling level is preserved or enhanced. After performing the allowed exchanges, some items may remain free. These items are reinserted using the FFD heuristic, completing the solution in a feasible and consistent manner.

For this study, all the knowledge generated from phases 1–6 was leveraged; that is, the GGA-CGT+GLX-1+FI/R with the original parametrization, together with the proposed adaptive control scheme for the parameter n_c, was used. In this way, the FFD strategy was replaced with the RP-EXP hueristic, and the resulting operator, denoted as CGT+GLX-1+FI/R+RP-EXP, was applied to the full benchmark set. Unless otherwise stated, all experiments described in this section were conducted using the baseline parameter configuration summarized in Table 3, except for the parameter n_c.

To facilitate understanding of the proposed crossover and repair mechanism, a practical example of GLX-1+FI/RP-EXP operation is presented in Figure 7, illustrating the genetic transmission process, the removal of bins with duplicated items, and the subsequent repair through controlled exchanges and reinsertion.

Table 8 presents the experimental results. As in Table 7, the first column reports the instance class, followed by the number of cases per class, the number of optima achieved, and the number of generations required. Lastly, the final four columns present descriptive statistics of the n_c values used for each instance class, including the average (mean), the minimum (min), the maximum (max), and the standard deviation (sd). In this table, n_c is reported as a normalized proportion in the range $[0, 1]$, representing the fraction of the population selected for crossover (i.e., values can be directly interpreted as percentages in the range $[0, 100]\%$).

The results from Table 8 show an improvement in the total number of optimal solutions, reaching 3622, which represents an increase of 52 solutions compared to the version without post-crossover RP-EXP. In addition, the average number of generations decreased from 168.70 to 160.98, indicating an improvement in algorithmic efficiency. Regarding the parameter n_c, a slight decrease was observed, which may indicate that the solutions generated by the crossover operator are being further improved by the RP-EXP method, particularly through better bin filling. At the class level, the use of RP-EXP provides notable improvements in data set 3, triplets, hard28, and the BPP.25–BPP.75 families. In contrast, for the BPPP1 class, a reduction in the number of optimal solutions is observed; however, this is accompanied by a lower number of generations required to reach them, suggesting a possible reduction in robustness for some instances but an increase in convergence speed for others.

Overall, the results indicate that combining the adaptive control scheme for the parameter n_c with the repair strategy RP-EXP improves both the quality and efficiency of the

Algorithm 3: RP-EXP as a post-crossover repair procedure

Input: Current solution S, set of free items F
Generate a random permutation of the bins in S, obtaining a sequence of b bins $S = (B_1, \ldots, B_b)$;
Sort F in ascending order according to item weight;
$j \leftarrow 1$;
while $j \leq b$ **do**
  foreach *pair of items* (p, s) *in bin* B_j **do**
    $exchanged \leftarrow$ FALSE;
    foreach *pair of items* (i, k) *in* F **do**
      if $w_i \geq w_p + w_s$ ***and*** $Feasible(S' \leftarrow Swap(S, F, (p, s), i))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        $exchanged \leftarrow$ TRUE;
        break;
      end
      if $w_k \geq w_p + w_s$ ***and*** $Feasible(S' \leftarrow Swap(S, F, (p, s), k))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        $exchanged \leftarrow$ TRUE;
        break;
      end
      if $w_i + w_k \geq w_p + w_s$ ***and*** $Feasible(S' \leftarrow Swap(S, F, (p, s), (i, k)))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        $exchanged \leftarrow$ TRUE;
        break;
      end
      if $w_i \geq w_p$ ***and*** $Feasible(S' \leftarrow Swap(S, F, (p), i))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        $exchanged \leftarrow$ TRUE;
        break;
      end
      if $w_i \geq w_s$ ***and*** $Feasible(S' \leftarrow Swap(S, F, (s), i))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        $exchanged \leftarrow$ TRUE;
        break;
      end
      if $w_k \geq w_p$ ***and*** $Feasible(S' \leftarrow Swap(S, F, (p), k))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        $exchanged \leftarrow$ TRUE;
        break;
      end
      if $w_k \geq w_s$ ***and*** $Feasible(S' \leftarrow Swap(S, F, (s), k))$ **then**
        $S \leftarrow S'$;
        $j \leftarrow j + 1$;
        $exchanged \leftarrow$ TRUE;
        break;
      end
    end
    if $exchanged$ **then**
      continue;
    end
  end
end
Apply FFD to reinsert remaining free items in F into solution S;
end RP-EXP.

GGA-CGT. The increase in the number of optimal solutions, the reduction in the number of generations required to reach them, and the stabilization of the parameter n_c throughout the search indicate a more stable evolutionary process. These results further suggest that the solutions produced by the proposed crossover operator can be effectively reinforced through a more structured repair process. More broadly, they underscore the benefit of incorporating feedback from the crossover operator into the algorithm's decision-making process, indicating that similar improvements could be achieved by extending control mechanisms to other parameters of the GGA-CGT. This observation naturally motivates the exploration of additional parameter control strategies reported in the literature. The following subsection provides an overview of these approaches and discusses how they may complement, or contrast with, the proposed adaptive crossover mechanism.

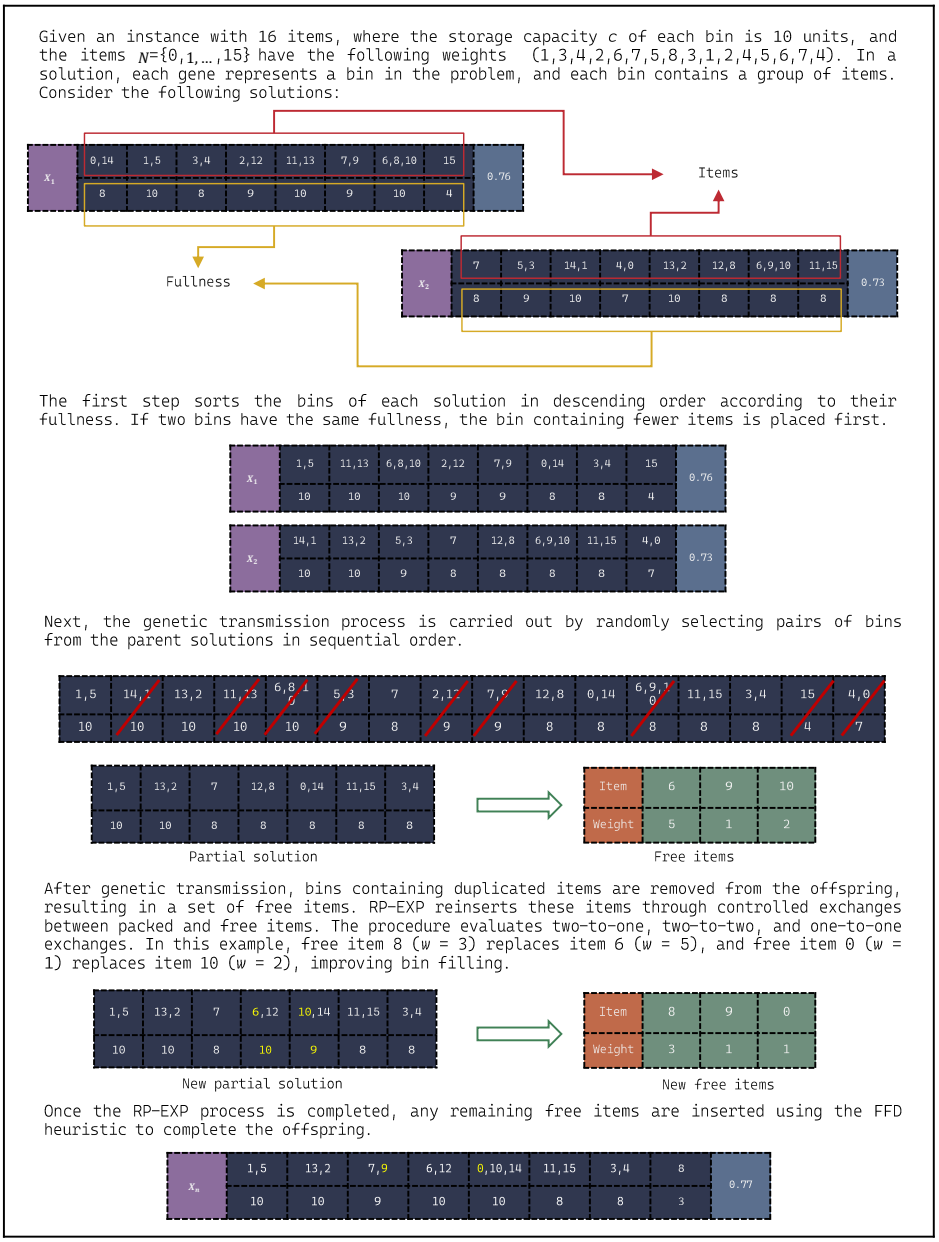


Figure 7. Schematic overview of the GLX-1+FI/R operator.

5.8. Phase 8: parameter control strategies in the GGA-CGT

Recent advances in evolutionary computation have emphasized the development of adaptive parameter-control mechanisms for GGAs, aimed at enhancing search stability and reducing sensitivity to initial settings. These approaches leverage feedback derived from population dynamics or operator-level performance to regulate algorithmic behavior, minimizing dependence on manual calibration. Against this backdrop, this subsection reviews the main parameter control strategies reported in the literature for GGAs, with particular emphasis on approaches that modify critical operators such as mutation, as well as parameters directly associated with its execution. The relationship between these strategies and the version of the algorithm proposed in this work is also discussed, which incorporates an adaptive control scheme for the crossover parameter n_c together with the repair operator RP+EXT. Finally, the potential synergies and distinctions between existing

Class	Cases	Opt.	Gen.	n_c			
				Mean	min	max	sd
data set 1	720	718	132.26	0.89	0.66	1.00	0.89
data set 2	480	480	24.55	0.84	0.62	1.00	0.84
data set 3	10	10	383.10	0.76	0.54	1.00	0.76
triplets	80	80	55.39	0.73	0.48	1.00	0.73
uniform	80	79	14.51	0.84	0.52	1.00	0.84
hard28	28	11	459.00	0.94	0.38	1.00	0.94
was1	100	100	0.70	0.90	0.80	1.00	0.90
was 2	100	100	185.37	0.95	0.80	1.00	0.95
gau 1	17	16	105.94	0.78	0.50	1.00	0.78
BPP.25	700	511	178.41	0.70	0.44	1.00	0.70
BPP.5	700	404	241.31	0.82	0.52	0.96	0.82
BPP.75	700	507	203.04	0.98	0.90	1.00	0.98
BPPP1	700	606	109.20	0.92	0.78	1.00	0.92
Total	4415	3622	160.98	0.85	0.61	1.00	0.85

Table 8. Overall results of the GGA-CGT+GLX1+FI/R+ RP-EXT.

parameter control schemes and the adaptive control scheme introduced in Phase 6 for the parameter n_c were analyzed. Several parameter control strategies for numerical and categorical parameters have been reported in the evolutionary computation literature. In particular, [8] presents an experimental study of parameter control strategies for the GGA-CGT. In that work, different rules were evaluated to determine which ones were best suited for each parameter of the algorithm. The parameters analyzed were n_m, k_{ns}, and k_{cs}. The control function used to update the mutation parameter n_m was originally proposed in [14]. This rule computes the accumulated average of the proportion of individuals with unique fitness values across generations, and is defined as:

$$aux_n_m = \frac{1}{g}\sum_{i=1}^{g}\frac{n_i^*}{|P|}, \tag{11}$$

where $|P|$ is the population size, n_i^* is the number of individuals with unique fitness values in generation i, and g denotes the current generation. With this formulation, the value of aux_n_m decreases more rapidly when the number of individuals with repeated fitness values increases, whereas the decrease is slower when diversity is high. Thus, the rule adapts the mutation level according to the historical behavior of population diversity.

However, this function presents a drawback: in some generations, aux_n_m may approach or even reach 1.0, which is not desirable because the algorithm maintains an elite group whose individuals must not be modified. To address this issue, [8] proposed a corrected version that imposes lower and upper bounds on the original function, ensuring that the number of individuals selected for mutation remains valid. The adjusted rule is defined as:

$$n_m = \lfloor aux_n_m \cdot (|P| - |B|) \rceil, \tag{12}$$

where $|P|$ is the population size, and $|B|$ is the size of the elite group. The intermediate value aux_n_m is bounded as follows:

$$aux_n_m = \begin{cases} l_{\text{inf}}, & \text{if } aux_n_m < l_{\text{inf}}, \\ l_{\text{sup}}, & \text{if } aux_n_m > l_{\text{sup}}, \\ aux_n_m, & \text{otherwise.} \end{cases} \tag{13}$$

with limits $l_{\text{inf}} = 0.1$ and $l_{\text{sup}} = 0.9$. This modification ensures that n_m remains within a controlled and feasible range consistent with the constraints of the GGA-CGT.

For the parameter that regulates the rate of change in cloned solutions (k_{cs}), the update function used was originally introduced in [53] as a control mechanism for crossover and mutation rates. Later, [8] identified this function as the most suitable option for the parameter k_{cs}. The rule is defined as:

$$k_{cs} = r_0 + (r_1 - r_0)\,\frac{g}{max_gen}, \tag{14}$$

where r_0 and r_1 denote the initial and final parameter values, g is the current generation, and max_gen is the maximum number of generations of the algorithm. This function produces a linear increase in k_{cs} over the evolutionary process.

For the parameter k_{ns}, which controls the rate of change in non-cloned solutions, the function proposed in [13] was employed:

$$k_{ns} = \left(r_0 + (r_1 - r_0)\,\frac{g}{max_gen}\right) \cdot \cos^2(C \cdot g), \tag{15}$$

where r_0, r_1, g, and max_gen have the same meaning as in the previous equation, and C is a scaling factor that adjusts the frequency of the oscillatory component. This formulation introduces a nonlinear modulation that amplifies or attenuates the rate of change along the execution while maintaining the same overall scale as the original linear function.

As part of this final experimental exploration phase, the GGA-CGT was implemented with the operator GLX-1, the heuristic strategies FI/R, the repair operator RP-EXT, and the scheme proposed to control the parameter n_c. Thus, the parameter control rules for mutation and for the k parameters described above were incorporated into the algorithm. Although the base metaheuristic remains GGA-CGT, the resulting version is now almost fully adaptive and integrates heuristic decision strategies within the GLX-1 operator, guided by the Fullness and Items criteria. Given the incorporation of these adaptive and heuristic components, the proposed algorithm was named Adaptive Grouping Genetic Algorithm with Controlled Gene Transmission based on Fullness–Items Strategies (AGGA-CGT-FIS), a designation that reflects the methodological consolidation achieved in this work. In this way, the AGGA-CGT-FIS was applied to the full benchmark set.

The remaining parameters were kept fixed across all configurations, i.e., $|P| = 100$, $|B| = 10$, $max_gen = 500$, $life_span = 10$. For the control parameters r_0 and r_1, the values established in previous studies were adopted. In particular, for k_{cs} the values were $r_0 = 13.5$ and $r_1 = 4.5$, while for k_{ns} the values were $r_0 = 7.5$ and $r_1 = 1.5$. In both cases, the corresponding functions are decreasing, allowing the intensity of the operators to gradually reduce as the evolutionary process progresses.

Table 9 presents the results obtained by the AGGA-CGT. It begins with the instance class, followed by the number of instances, the optima achieved, and the required number of generations. The final four columns summarize the n_c values per class through their mean, minimum, maximum, and standard deviation. In this table, n_c is reported as a normalized proportion in the range $[0, 1]$, representing the fraction of the population selected for crossover (i.e., values can be directly interpreted as percentages in the range $[0, 100]\%$).

The results in Table 9 indicate that, overall, there is an increase in the number of optimal solutions obtained, together with a reduction in the average number of generations required. Additionally, the parameter n_c exhibits stable behavior, with consistently high values and low variability, suggesting that the adaptive mechanism achieves an adequate balance between exploration and exploitation. Improvements are particularly notable in the

Class	Cases	Opt.	Gen.	n_c			
				Mean	min	max	sd
data set 1	720	718	132.64	0.96	0.86	1	0.96
data set 2	480	480	25.02	0.93	0.78	1	0.93
data set 3	10	9	425.40	0.91	0.74	1	0.91
triplets	80	80	74.00	0.90	0.70	1	0.90
uniform	80	79	14.19	0.94	0.72	1	0.94
hard28	28	6	492.96	0.97	0.62	1	0.97
was1	100	100	0.95	0.95	0.90	1	0.95
was 2	100	100	188.32	0.98	0.90	1	0.98
gau 1	17	16	107.24	0.91	0.78	1	0.91
BPP.25	700	550	167.55	0.87	0.76	1	0.87
BPP.5	700	453	220.89	0.94	0.74	0.98	0.94
BPP.75	700	568	149.65	1.00	0.96	1	1.00
BPPP1	700	641	77.39	0.97	0.88	1	0.97
Total	4415	3800	159.71	0.94	0.80	1.00	0.94

Table 9. Overall performance of the AGGA-CGT-FIS.

data set 3, triplets, BPP.25–BPP.75, and BPPP1 classes, while more challenging sets such as hard28 still present opportunities for further refinement. Altogether, these results indicate that combining adaptive control with the repair operator RP-EXT effectively enhances the quality and stability of the AGGA-CGT-FIS performance. It is important to note that the proposed crossover control scheme is directly linked to the parameter k_{cs}, which is involved in the computation of the accumulated crossover value defined in Equation (10). In this formulation, k_{cs} regulates the relative influence of each generation on the averaged value of n_c and therefore modulates the intensity of the crossover operator. In the proposed version of the algorithm, k_{cs} is no longer a fixed parameter; instead, it is dynamically updated according to the rule presented in Equation (14). Since this function decreases over time, the parameter k_{cs} takes larger values in the early stages and smaller values in later generations, altering the sensitivity of the parameter n_c calculation. This time-dependent adjustment can significantly influence the balance between exploration and exploitation by modifying how genetic transmission is weighted throughout the evolutionary process.

To better understand this behavior, an additional study focused on analyzing the algorithm's search dynamics was conducted. In this analysis, nine indicators, previously defined in Phase 5, describing the search process were monitored: *Best*, *Worst*, *Average*, *Repeated*, *Average ratio*, *Free items*, *Partial bins*, *Best crossover ratio*, and *Worst crossover ratio*. The same seven instances were used, and the results are reported using the same presentation format. Figure 8 summarizes the overall behavior of the AGGA-CGT-FIS. As in Figure 6, each row of plots corresponds to a different instance and consists of four graphs: (1) performance-related indices characterizing the overall behavior of AGGA-CGT-FIS; (2) metrics assessing the degree of bin filling in solutions generated by the crossover operator; (3) the crossover ratios of the best and worst solutions under the proposed adaptive parameter control scheme; and (4) the evolution of the mutation parameters k_{cs} and k_{ns}, represented by pink and blue lines, respectively.

Figure 8 provide insight into the combined dynamics of performance, solution structure, and crossover behavior under the adaptive control scheme and the employed repair method. It is particularly interesting to examine the behavior of the worst solution in instances such as N3c3w4_c, t120_10, and hBPP195, where a clearly oscillatory pattern is observed throughout the generations. In contrast, the instances belonging to the BPPv_{u}_c classes exhibit more stable and less fluctuating behavior. Additionally, the proportion of individuals with repeated fitness values remains relatively low in almost all cases (below 0.2), suggesting that the online parameter adjustments help maintain adequate levels of

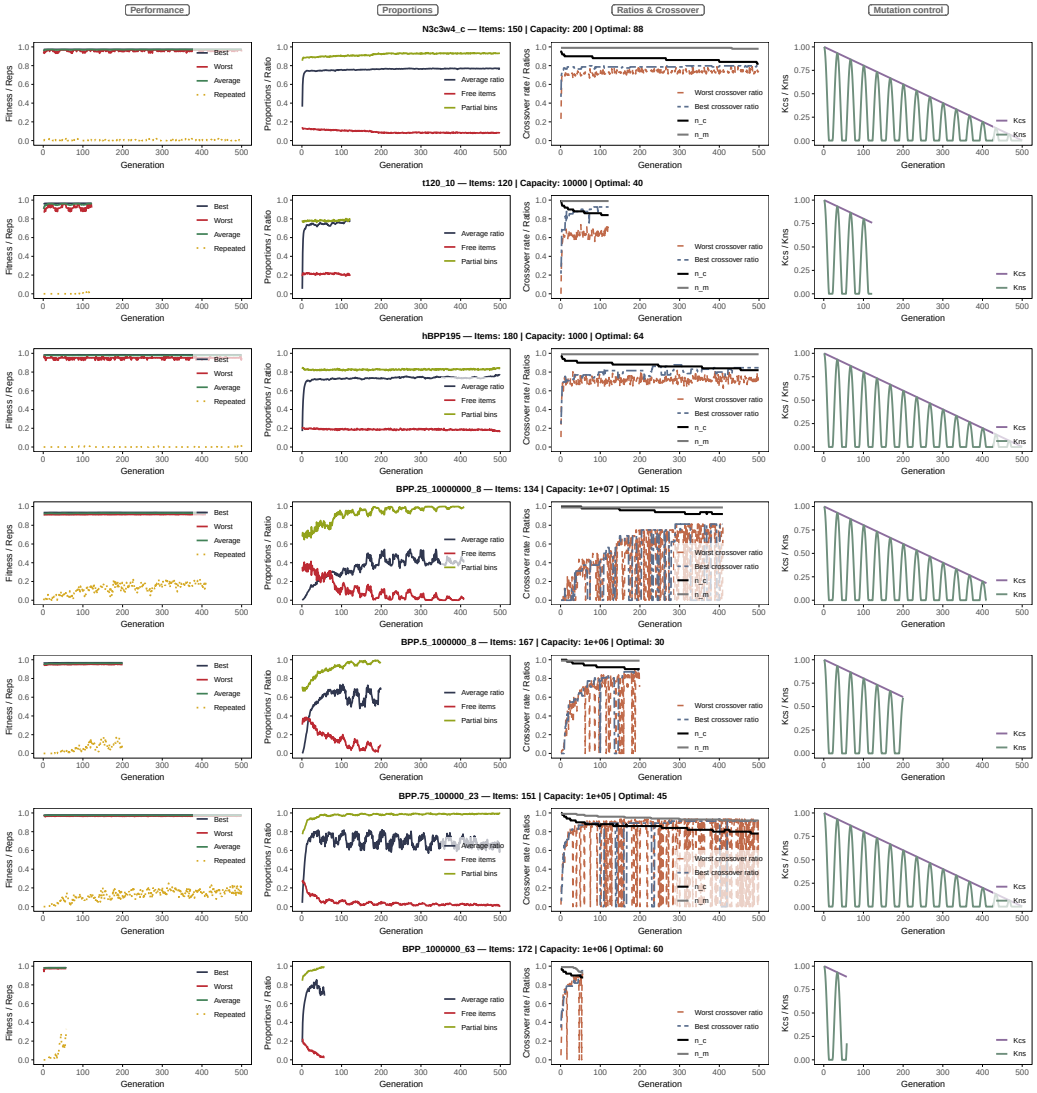


Figure 8. Algorithmic behavior of the AGGA-CGT-FIS.

diversity during the search. Regarding the behavior after crossover, an oscillatory pattern is also observed, especially in the triplets and BPPv_u_c classes. This effect may be largely influenced by the adaptive parameter dynamics, where certain parameter variations could exert a stronger effect on the behavior of the crossover operator. The plots corresponding to the best and worst solutions show a different trend compared to the previous cases. The best solution may still contain bins that are not fully filled, but their number decreases noticeably as the generations progress. This indicates that the adaptive control strategy is having a positive impact on the structural quality of the solutions. As for the parameter n_c, it remains above 60% in most instances, gradually decreasing toward the final generations, which is consistent with the expected behavior of the proposed control scheme.

Additionally, the behavior of the parameters k_{cs} and k_{ns} was analyzed, whose normalized values are shown in the last column of the panel. The parameter k_{cs} regulates the sensitivity of the parameter n_c through the exponent $1/k_{cs}$ and exhibits a clear relationship with the dynamics of the crossover operator. Higher values of k_{cs} increase n_c, promoting a more exploratory search, whereas lower values decrease n_c, shifting the process toward exploitation of promising regions. This tendency is especially apparent in the BPP.25, BPP.5, and BPP.75 classes, as well as in the triplets set, where fluctuations in k_{cs} are directly

reflected in the crossover ratios (best and worst). In contrast, instances such as hard28 and data set 1 display a more stable behavior, with both k_{cs} and n_c evolving smoothly across generations due to the reduced structural variability in the proportion of incomplete bins r_i.

Lastly, the final generations exhibit a stable behavior in terms of the average number of bins filled to their maximum capacity and the number of freed items. This stability is the result not only of the crossover control imposed by k_{cs}, but also of the RP-EXT repair mechanism. RP-EXT restructures the offspring after crossover, correcting inconsistencies and guiding solutions toward fuller configurations, which reduces both the number of free items and the proportion of partially filled bins. Consequently, the curves for free items and partial bins flatten and converge as the evolution progresses. Overall, the combined effect of the k_{cs}-based crossover control and the RP-EXT repair mechanism allows the algorithm to begin with strong exploration behavior. It then transitions naturally into a more stable and efficient exploitation phase. This process consolidates increasingly complete configurations within the population.

The patterns observed in Figure 8 show that the interaction among the adaptive crossover control scheme, the RP-EXP repair method, and the dynamic rules for mutation and k parameters modifies the algorithm's dynamics in a meaningful way. As a result, both stability and solution structure improve throughout the generations. However, to determine whether these internal improvements result in better performance compared to the original GGA-CGT, a direct comparison between both versions is necessary. The following section presents this comparative analysis, followed by a statistical evaluation to assess whether the observed differences are significant.

6. Comparison Between the GGA-CGT and the Proposed AGGA-CGT-FIS

After establishing the final version of the algorithm, a comparison between the GGA-CGT and the proposed AGGA-CGT-FIS was conducted. Table 10 presents the experimental results. The first two columns contain the class and the number of instances per class, respectively; columns three, four, and five present the number of optima, the number of generations, and the time, in seconds, used by the GGA-CGT, while columns six, seven, and eight show the same information for the proposed AGGA-CGT-FIS. All experiments were executed on a workstation equipped with an 11th Generation Intel® Core™ i7-1165G7 processor running at 2.80 GHz and 16 GB of RAM, using a 64-bit operating system.

Overall, AGGA-CGT-FIS achieves a higher number of optimal solutions in nearly all classes. The only notable exception is the hard28 class, where ten fewer optimal solutions are obtained. However, given the small size of this class (28 instances), this reduction represents a large percentage difference but does not alter the overall performance trend. Similarly, in classes such as data set 1, data set 3, and uniform, the difference between two or a single optimal solution is negligible when considering the size of these sets.

The most substantial improvements are observed in the BPPv_u_c classes (BPP.25, BPP.5, BPP.75, and BPP1), where the number of optimal solutions increases consistently by more than one hundred instances, together with a significant reduction in the average number of generations. At a global level, the proposed algorithm increases the proportion of optimal solutions from 71.4% to 86.1%, corresponding to an improvement of 14.7% over the full benchmark.

These gains are particularly pronounced in challenging classes such as BPP.75, which exhibit increased structural complexity. In this class, optimal solutions typically require bins to be filled very close to their maximum capacity, while the presence of items with weights up to $0.75c$ imposes stricter packing constraints. Under these conditions, small

Class	Cases	GGA-CGT			AGGA-CGT-FI		
		Opt.	Gen.	Time	Opt.	Gen.	Time
data set 1	720	720	131.96	0.24	718	132.64	3.00
data set 2	480	480	24.58	0.09	480	25.02	1.74
data set 3	10	10	361.80	1.20	9	425.40	12.51
triplets	80	80	40.48	0.34	80	74.00	2.07
uniform	80	80	10.94	0.25	79	14.19	1.80
hard28	28	16	385.46	1.77	6	492.96	8.43
was1	100	100	0.67	0.01	100	0.95	0.02
was 2	100	100	185.24	0.70	100	188.32	2.83
gau 1	17	16	105.71	0.16	16	107.24	0.76
BPP.25	700	448	225.72	4.47	550	167.55	10.51
BPP.5	700	321	296.72	2.63	453	220.89	6.31
BPP.75	700	276	333.21	1.96	568	149.65	2.62
BPPP1	700	504	219.64	1.17	641	77.39	1.07
Total	4415	3151	178.63	1.71	3800	159.71	4.15

Table 10. Comparison of the GGA-CGT vs GGA-CGT-FI performance.

changes in item grouping and bin utilization can have a substantial impact on solution quality.

Preliminary analyses further indicate that instances in this class tend to exhibit a faster increase in the number of solutions with repeated fitness values, reflecting an early tendency toward reduced population diversity. The integration of adaptive control mechanisms within the variation operators helps mitigate this behavior. In particular, the mutation control scheme adjusts its intensity based on the population's fitness distribution, reducing mutation pressure when repeated fitness values become dominant. In parallel, the proposed crossover strategy, combined with the RP-EXP repair mechanism, introduces more disruptive yet structured changes by explicitly improving bin filling after genetic transmission. This combination enables the algorithm to better reorganize items under tight capacity constraints, leading to more efficient packings and explaining the stronger performance gains observed in the BPP.75 and BPP1 classes.

From a computational perspective, these additional adaptive mechanisms introduce a moderate increase in execution time. An average increase of 2.44 seconds is observed in AGGA-CGT-FI when compared to GGA-CGT. However, this increase should not be interpreted as an inherent drawback of the proposed approach, but rather as a direct consequence of the additional control strategies incorporated into the algorithm. While GGA-CGT applies crossover to only 20% of the population, the proposed method dynamically regulates the crossover process, reaching n_c values above 70% of the population in this case.

Moreover, AGGA-CGT-FI incorporates the adaptive control of four parameters, including parameters such as k, which regulate the rate of change and vary dynamically within intervals that may reach higher values than those used as fixed parameters in GGA-CGT. As a consequence, the mutation operator performs a larger number of internal operations per generation, including both the application of the operator itself and its associated repair procedure. In addition, the new crossover repair scheme integrates the RP heuristic as an improvement mechanism, introducing further computational effort. Taken together, these elements explain the observed increase in the computational cost of the proposed approach. Nevertheless, this overhead remains bounded and is consistent with the improved adaptive behavior of the algorithm and the higher quality of the solutions obtained.

To validate these results, a global analysis was conducted to determine whether the differences between GGA-CGT and AGGA-CGT-FIS are statistically significant. For this study, 30 independent runs with different random seeds were performed for each algorithm, using the total number of optimal solutions as the performance metric. Next, the Shapiro–Wilk test was applied to assess the normality of the data, indicating that both samples follow a normal distribution ($p > 0.05$). Subsequently, Levene's test was conducted to evaluate the homogeneity of variances, confirming that the assumption of homoscedasticity holds for the two samples, as no statistically significant differences in variance were detected ($p > 0.05$).

Since both assumptions were satisfied, a two-sample t-test was applied. The results showed a p-value of $< 2.2 \times 10^{-16}$ and a mean difference of approximately 658 optimal solutions, confirming that the improvement achieved by AGGA-CGT-FIS is statistically robust. Table 11 presents the descriptive statistics (mean and standard deviation) obtained from the 30 runs of each algorithm, reinforcing the substantial performance gap.

Algorithm	Mean	SD
AGGA-CGT-FIS	3785.40	9.74
GGA-CGT	3127.83	9.93

Table 11. Descriptive statistics (mean and standard deviation) of optimal solutions obtained by both algorithms over the full benchmark set.

Beyond the global comparison, a detailed per class analysis was conducted to assess whether the performance differences between the original GGA-CGT and the proposed AGGA-CGT-FIS are consistent across all instance families or concentrated in specific groups. For each class, 30 independent runs were executed using different random seeds, and the performance metric used was the number of optimal solutions obtained per run.

Figure 9 shows normalized boxplots in the interval $[0, 1]$ for each class, allowing for a clear visual comparison. In most small or structurally simple classes (data set 1, data set 2, data set 3, triplets, uniform, was 1, was 2, and gau 1), both algorithms achieve values very close to the maximum, resulting in minimal or nonexistent differences. This is reflected in the boxplots, where the distributions appear nearly collapsed at the upper bound.

In contrast, the BPPv_u_c classes (BPP.25, BPP.5, BPP.75, and BPPP1) exhibit clear differences. The boxplots show that AGGA-CGT-FIS not only shifts the median toward higher values but also reduces variability and concentrates the distribution near the upper range, consistent with the improvements reported in Table 10. For these classes, the proposed method consistently outperforms the original algorithm.

Table 12 summarizes the statistical tests performed for each class. Depending on the normality and homoscedasticity assumptions, either a two-sample t-test or a Wilcoxon rank-sum test was applied. The results reveal two clear patterns.

First, in six classes data set 1, data set 3, triplets, uniform, hard28, and gau 1, the original GGA-CGT performs significantly better, as indicated by low p-values favoring the baseline algorithm. In was 2, no statistically significant differences were detected, and in data set 2 and was 1, both algorithms produce identical results.

Second, for all classes associated with the BPPv_u_c classes (BPP.25, BPP.5, BPP.75, and BPP1), the proposed AGGA-CGT-FIS shows statistically significant improvements, with p-values far below the 0.05 significance level. This matches the substantial increases in the number of optimal solutions reported previously for these classes.

Overall, the per-class analysis shows that AGGA-CGT-FIS consistently improves performance in the structured and challenging BPPv_u_c classes, where the adaptive control mechanisms (crossover, mutation, and k parameters) and the RP-EXP repair method contribute positively. Conversely, in simpler or more irregular classes, the original GGA-CGT

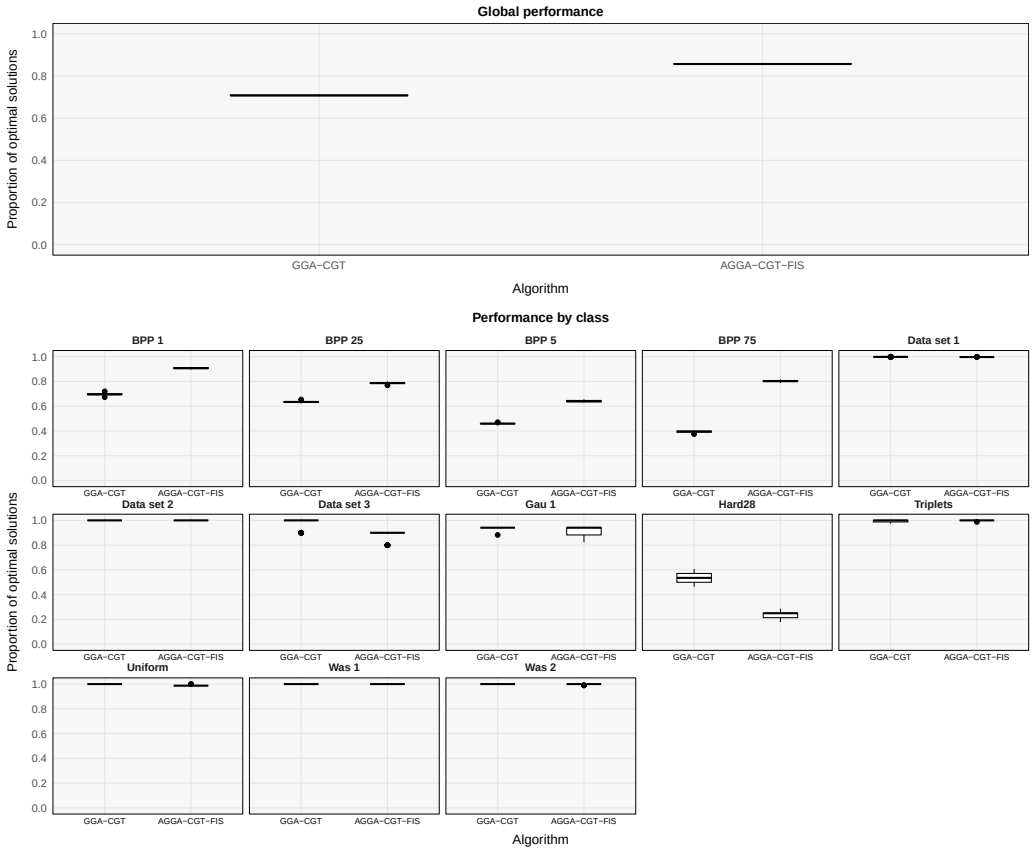


Figure 9. Boxplots of global and per-class performance for GGA-CGT and AGGA-CGT-FIS. The figure shows the distribution of the number of optimal solutions obtained across 30 independent runs for each algorithm, allowing a uniform comparison of variability and central tendency across all benchmark classes.

remains more effective, suggesting that the benefits of the proposed control scheme depend strongly on the structural characteristics of each instance class.

7. Conclusions and future work

In this work, the Adaptive Grouping Genetic Algorithm with Controlled Gene Transmission based on Fullness–Items Strategies (AGGA-CGT-FIS) was introduced. The proposed approach builds upon the state-of-the-art GGA-CGT framework for solving the one-dimensional Bin Packing Problem (1D-BPP) and extends it through several methodological and algorithmic enhancements. The main contributions include the incorporation of a novel crossover operator inspired by the GLX-1 operator, enriched with strategies for ordering parental genes and controlling genetic material transmission according to Fullness and Item-based criteria. In addition, adaptive control schemes were introduced for key evolutionary parameters associated with both crossover and mutation operators.

To systematically develop and validate the proposed approach, an eight-phase experimental framework was designed. The first phase explores the performance of the original GGA-CGT using multiple state-of-the-art operators. The subsequent two phases investigate alternative strategies for ordering parental genes and regulating gene transmission. This is followed by a dedicated phase analyzing the behavior of GGA-CGT with the proposed FI-GLX operator, using nine search-process indicators and visualized through descriptive plots. Next, an adaptive control scheme for the n_c parameter was introduced and its impact on GGA-CGT performance was evaluated. A further phase assessed the effectiveness

Class	Cases	Test	p-value	Best
Data set 1	720	Wilcoxon	9.81×10^{-7}	GGA-CGT
Data set 2	480	N/A	–	Equal data
Data set 3	10	Wilcoxon	5.44×10^{-10}	GGA-CGT
Triplets	80	Wilcoxon	0.002953	GGA-CGT
Uniform	80	Wilcoxon	7.15×10^{-13}	GGA-CGT
Hard28	28	Wilcoxon	1.57×10^{-11}	GGA-CGT
Was 1	100	N/A	–	Equal data
Was 2	100	Wilcoxon	0.0814	No differences
Gau 1	17	Wilcoxon	0.0002827	GGA-CGT
BPP.25	700	t-test	$< 2.2 \times 10^{-16}$	AGGA-CGT-FIC
BPP.5	700	Wilcoxon	2.84×10^{-11}	AGGA-CGT-FIC
BPP.75	700	Wilcoxon	2.83×10^{-11}	AGGA-CGT-FIC
BPPP1	700	t-test	$< 2.2 \times 10^{-16}$	AGGA-CGT-FIC

Table 12. Results of the statistical tests comparing the number of optimal solutions obtained by the original GGA-CGT and the proposed AGGA-CGT-FIS.

of a novel repair strategy, termed RP-EXT. Finally, in the last phase, additional adaptive schemes were incorporated to regulate parameters associated with the mutation operator.

As a result of the application of this eight-phase experiential framework, the AGGA-CGT-FIS was developed as an enhanced version of the GGA-CGT that explicitly incorporates problem-domain knowledge. To demonstrate the effectiveness of the proposed algorithm, a comprehensive set of statistical tests was conducted to assess whether the performance improvements achieved by AGGA-CGT-FIS are statistically significant when compared with the baseline GGA-CGT.

Statistical tests showed that the proposed operator FI-GLX-1 and control mechanisms significantly improve performance in the structured and demanding BPPv_u_c classes, where AGGA-CGT-FIS achieves higher numbers of optimal solutions and requires fewer generations on average. These improvements were confirmed statistically, with highly significant p-values across all BPPv_u_c classes. Conversely, for smaller or structurally irregular instance classes, the original GGA-CGT remains more effective, indicating that the benefits of the proposed scheme depend on the underlying structure of each problem family.

Overall, the results show that FI-GLX-1 and the adaptive parameter control strategies substantially enhance the effectiveness of the GGA-CGT, particularly in grouping problems where the solution structure can be effectively exploited, such as the BPPv_u_c classes. The combination of the new crossover operator, the adaptive crossover control scheme, and the RP-EXP repair method yields a robust and competitive approach for solving the 1D-BPP.

From a computational perspective, the proposed enhancements introduce a moderate increase in execution time when compared to the baseline GGA-CGT. This overhead arises primarily from the incorporation of adaptive control mechanisms and the higher proportion of individuals selected for crossover during the evolutionary process, as well as from the use of dynamically adjusted k values that lead the mutation operator to perform additional internal operations and repair steps. In addition, the crossover operator now incorporates a pairwise reorganization mechanism together with the RP heuristic, which further increases the computational effort. However, this increase remains bounded and is directly associated with the improved adaptive behavior of the algorithm and the higher quality of the solutions obtained. As such, the observed computational cost represents a reasonable trade-off for the significant gains in robustness and performance achieved by AGGA-CGT-FIS on structured and demanding 1D-BPP instances.

Despite the observed improvements, some limitations of the proposed approach should be acknowledged. The adaptive control mechanisms introduced in AGGA-CGT-FIS rely on indicators extracted from the evolutionary behavior of the population, such as fitness diversity and structural properties of the solutions. Consequently, their effectiveness is closely linked to how these indicators evolve during the search process.

If problem instances exhibit markedly different characteristics, such as highly homogeneous or very small item weights, low weight variability, or alternative item–capacity distributions, the behavior of these indicators may change, potentially affecting the responsiveness of the adaptive schemes. In such scenarios, the balance between exploration and exploitation induced by the proposed control strategies may require further adjustment to remain effective.

As future work, several promising directions can be pursued. First, a more detailed analysis of the internal behavior of FI-GLX-1 and the adaptive control mechanisms could provide deeper insight into how these components influence convergence, diversity, and structural preservation throughout the evolutionary process. Understanding these dynamics may help refine the operator or identify the conditions under which it is most effective.

An immediate extension of this work would be to assess the behavior of AGGA-CGT-FIS on additional 1D-BPP instances with distinct structural characteristics, in order to analyze how variations in item distributions and packing constraints influence the effectiveness of the proposed adaptive mechanisms. Such studies would help assess the robustness of the proposed adaptive mechanisms under diverse evolutionary dynamics and further clarify the conditions under which the algorithm performs most effectively.

In addition, the adaptive control strategies could be extended to regulate further parameters of the algorithm, or replaced with more advanced control strategies, such as feedback rules grounded in more expressive diversity metrics. These extensions may enable more precise and stable adaptation and improved robustness across different problem families. Evaluating the proposed approach on other grouping problems or applied domains would further help assess its generality and practical relevance.

Beyond the one-dimensional setting, future research could explore the extension of the proposed ideas to more complex variants of the bin packing problem, such as multi dimensional or multi constraint BPPs, where structural information plays an even more critical role. Finally, applying the proposed adaptive framework to real-world packing and allocation problems may provide valuable insights into its practical applicability and scalability in operational environments.

Author Contributions: Conceptualization, S.A.-L., M.Q.-C., O.R.-F., and A.G.-H.; methodology, S.A.-L. and M.Q.-C.; software, S.A.-L. and M.Q.-C.; validation, S.A.-L. and M.Q.-C.; formal analysis, S.A.-L. and M.Q.-C.; investigation, S.A.-L., M.Q.-C., and A.G.-H.; resources, S.A.-L.; writing—original draft preparation, S.A.-L. and O.R.-F.; writing—review and editing, S.A.-L., M.Q.-C., and O.R.-F.; visualization, S.A.-L.; supervision, M.Q.-C., O.R.-F., and A.G.-H.; project administration, M.Q.-C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The datasets generated and analyzed during the current study are not publicly available due to their size and the specialized nature of the experimental setup, but are available from the corresponding author upon reasonable request via email.

Conflicts of Interest: The authors declare no conflict of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript, or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

1D-BPP	One-Dimensional Bin Packing Problem
GGA-CGT	Grouping Genetic Algorithm with Controlled Gene Transmission
GGA	Grouping Genetic Algorithm
FF	First Fit
FFD	First Fit Decreasing
GLX	Gene-Level Crossover
UX	Uniform Crossover
ESX	Exon Shuffling Crossover
GPX	Greedy Partition Crossover
GLX-1	Gene-Level Crossover 1
FI-GLX-1	Fullness–Items Gene-Level Crossover 1
AGGA-CGT-FIS	Adaptive Grouping Genetic Algorithm with Controlled Gene Transmission based on Fullness-Items Strategies
RP	Rearrangement by Pairs
RP-EXP	Rearrangement by Pairs Expanded

References

1. Martello, S. *Knapsack problems : algorithms and computer implementations*; J. Wiley & Sons: Chichester New York, 1990.
2. Falkenauer, E.; Delchambre, A. A genetic algorithm for bin packing and line balancing. In Proceedings of the Proceedings 1992 IEEE International Conference on Robotics and Automation, Los Alamitos, CA, USA, may 1992; pp. 1186–1192. https://doi.org/10.1109/ROBOT.1992.220088.
3. Ramos-Figueroa, O.; Quiroz-Castellanos, M.; Mezura-Montes, E.; Schütze, O. Metaheuristics to solve grouping problems: A review and a case study. *Swarm and Evolutionary Computation* **2020**, *53*, 100643. https://doi.org/https://doi.org/10.1016/j.swevo.2019.100643.
4. Quiroz-Castellanos, M.; Cruz-Reyes, L.; Torres-Jimenez, J.; Gómez S., C.; Huacuja, H.J.F.; Alvim, A.C. A Grouping Genetic Algorithm with Controlled Gene Transmission for the Bin Packing Problem. *Comput. Oper. Res.* **2015**, *55*, 52–64. https://doi.org/10.1016/j.cor.2014.10.010.
5. Falkenauer, E. A hybrid grouping genetic algorithm for bin packing. *Journal of heuristics* **1996**, *2(1)*, 5–30. https://doi.org/https://doi.org/10.1007/BF00226291.
6. Amador-Larrea, S.; Quiroz-Castellanos, M.; Hoyos-Rivera, G.D.J.; Mezura-Montes, E. An Experimental Study of Grouping Crossover Operators for the Bin Packing Problem. https://doi.org/10.13053/CyS-26-2-4249.
7. Ramos-Figueroa, O.; Quiroz-Castellanos, M. An experimental approach to designing grouping genetic algorithms. *Swarm and Evolutionary Computation* **2024**, *86*, 101490. https://doi.org/https://doi.org/10.1016/j.swevo.2024.101490.
8. Barojas Vázquez, A. Estudio experimental de operadores de mutación para el algoritmo genético de agrupación con transmisión de genes controlada para el problema de empacado de objetos en contenedores de una dimensión. Tesis de maestría, Universidad Veracruzana, Xalapa, Veracruz, México, 2026.
9. Ramos-Figueroa, O.; Quiroz-Castellanos, M.; Mezura-Montes, E.; Cruz-Ramírez, N. An Experimental Study of Grouping Mutation Operators for the Unrelated Parallel-Machine Scheduling Problem. *Mathematical and Computational Applications* **2023**, *28*. https://doi.org/10.3390/mca28010006.
10. Fernández-Solano, O. Un estudio experimental de operadores de mutación para el algoritmo genético de agrupación para la segmentación de imágenes en RGB. Tesis de maestría, Universidad Veracruzana, Veracruz, México, 2023.
11. Carmona-Arroyo, G. A Grouping Genetic Algorithm for Variable Decomposition in Large-Scale Constrained Optimization Problems. Master's thesis, Universidad Veracruzana, 2024.
12. Zavaleta-García, N.A. Estudio experimental de operadores genéticos de cruza para el problema de segmentación de imágenes en color. Tesis de maestría, Universidad Veracruzana, Veracruz, México, 2023.
13. Flores-Torres, L.; Quiroz-Castellanos, M.; Ramos-Figueroa, O.; Amador-Larrea, S. An experimental approach to design deterministic and adaptive control schemes for grouping genetic algorithms. *Neural Computing and Applications* **2025**. https://doi.org/10.1007/s00521-025-11270-x.
14. Amador-Larrea, S.; Quiroz-Castellanos, M.; Ramos-Figueroa, O. An Experimental Study of Strategies to Control Diversity in Grouping Mutation Operators: An Improvement to the Adaptive Mutation Operator for the GGA-CGT for the Bin Packing Problem. *Mathematical and Computational Applications* **2025**, *30*. https://doi.org/10.3390/mca30020031.

15. Fleszar K., .C.C. TAverage-weight-controlled bin-oriented heuristics for the one-dimensional bin-packing problem. *European Journal of Operational Research* **2011**, *210(2)*, 176–184. https://doi.org/10.1016/j.ejor.2010.11.004.
16. Delorme, M.; Iori, M.; Martello, S. Bin packing and cutting stock problems: Mathematical models and exact algorithms. *European Journal of Operational Research* **2016**, *255*, 1–20. https://doi.org/https://doi.org/10.1016/j.ejor.2016.04.030.
17. Brandão, F.; Pedroso, J.P. Bin packing and related problems: General arc-flow formulation with graph compression. *Computers & Operations Research* **2016**, *69*, 56–67. https://doi.org/https://doi.org/10.1016/j.cor.2015.11.009.
18. Wei, L.; Luo, Z.; Baldacci, R.; Lim, A. A new branch-and-price-and-cut algorithm for one-dimensional bin-packing problems. *INFORMS Journal on Computing* **2020**, *32*, 428–443. https://doi.org/https://doi.org/10.1287/ijoc.2018.0867.
19. Baldacci, R.; Coniglio, S.; Cordeau, J.F.; Furini, F. A numerically exact algorithm for the bin-packing problem. *INFORMS Journal on Computing* **2024**, *36*, 141–162. https://doi.org/https://doi.org/10.1287/ijoc.2022.0257.
20. Alvim, A.C.F.; Ribeiro, C.C.; Glover, F.; Aloise, D.J. A Hybrid Improvement Heuristic for the One-Dimensional Bin Packing Problem. *Journal of Heuristics* **2004**, *10*, 205–229. https://doi.org/https://doi.org/10.1023/B:HEUR.0000026267.44673.ed.
21. Buljubašić, M.; Vásquez, M. Consistent neighborhood search for one-dimensional bin packing and two-dimensional vector packing. *Computers & Operations Research* **2016**, *76*, 12–21. https://doi.org/https://doi.org/10.1016/j.cor.2016.06.009.
22. González-San-Martín, J.E. Un estudio formal de heurísticas para el problema de empacado de objetos de una dimensión. Master's thesis, Instituto Tecnológico de Ciudad Madero, 2021.
23. Ali, A.I.; Keedwell, E.; Helal, A. A Differential Pheromone Grouping Ant Colony Optimization Algorithm for the 1-D Bin Packing Problem. In Proceedings of the Proceedings of the Genetic and Evolutionary Computation Conference, 2024, pp. 1463–1469.
24. Gezici, H.; Livatyali, H. Optimization of one-dimensional bin packing problem using a hybrid flower pollination algorithm. *Sigma: Journal of Engineering and Natural Sciences* **2023**, *41*.
25. Bai, R.; Błażewicz, J.; Burke, E.K.; et al. A simulated annealing hyper-heuristic methodology for flexible decision support. *4OR – A Quarterly Journal of Operations Research* **2012**, *10*, 43–66. https://doi.org/10.1007/s10288-011-0182-8.
26. Juárez, J.; Falcón-Cardona, J.G.; Ortiz-Bayliss, J.C. A steady state micro genetic algorithm for hyper-heuristic generation in one-dimensional bin packing. *Scientific Reports* **2025**, *15*, 27220.
27. Dósa, G.; éles, A.; Goswami, A.R.; Szalkai, I.; Tuza, Z. Solution of Bin Packing Instances in Falkenauer T Class: Not So Hard. *Algorithms* **2025**, *18*, 115.
28. Mutingi, M.; Mbohwa, C. Grouping Genetic Algorithms. *Advances and Applications. Switzerland: Springer International Publishing* **2017**, *243*.
29. Peddi, S.; Singh, A. Grouping genetic algorithm for data clustering. In Proceedings of the International Conference on Swarm, Evolutionary, and Memetic Computing. Springer, 2011, pp. 225–232.
30. Srinivas, M.; Patnaik, L.M. Adaptive probabilities of crossover and mutation in genetic algorithms. *IEEE Transactions on Systems, Man, and Cybernetics* **1994**, *24*, 656–667.
31. Bhatia, A.; Basu, S.K. Packing bins using multi-chromosomal genetic representation and better-fit heuristic. In Proceedings of the International Conference on Neural Information Processing. Springer, 2004, pp. 181–186.
32. Tan, B.; Ma, H.; Mei, Y. A group genetic algorithm for resource allocation in container-based clouds **2020**. pp. 180–196.
33. Vafaee, F.; Nelson, P.C. An explorative and exploitative mutation scheme. In Proceedings of the IEEE Congress on Evolutionary Computation, 2010, pp. 1–8. https://doi.org/10.1109/CEC.2010.5586142.
34. Mutingi, M.; Onwubolu, G.C. Integrated cellular manufacturing system design and layout using group genetic algorithms. *Manufacturing System, FA Aziz, Ed., InTech-Open* **2012**, pp. 205–222.
35. Mutingi, M.; Mbohwa, C. Home healthcare worker scheduling: a group genetic algorithm approach. In Proceedings of the Proceedings of the world congress on engineering, 2013, Vol. 1, pp. 3–5.
36. Mutingi, M.; Mbohwa, C. Task assignment in home health care: A fuzzy group genetic algorithm approach. In Proceedings of the The 25th annual conference of the Southern African institute of industrial engineering. Citeseer, 2013, pp. 1–10.
37. Mutingi, M.; Mbohwa, C. Home health care staff scheduling: Effective grouping approaches. In Proceedings of the IAENG Transactions on Engineering Sciences-Special Issue of the International Multi-Conference of Engineers and Computer Scientists, IMECS and World Congress on Engineering, CRC Press, Taylor & Francis Group, 2014, pp. 215–224.
38. Singh, K.; Sundar, S. A new hybrid genetic algorithm for the maximally diverse grouping problem. *International Journal of Machine Learning and Cybernetics* **2019**, *10*, 2921–2940.
39. Ramos-Figueroa, O.; Quiroz-Castellanos, M.; Mezura-Montes, E.; Kharel, R. Variation Operators for Grouping Genetic Algorithms: A Review. *Swarm and Evolutionary Computation* **2021**, *60*, 100796. https://doi.org/10.1016/j.swevo.2020.100796.
40. Fukunaga, A.S. A new grouping genetic algorithm for the Multiple Knapsack Problem. In Proceedings of the 2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence), 2008, pp. 2225–2232. https://doi.org/10.1109/CEC.2008.4631094.
41. Dokeroglu, T.; Cosar, A. Optimization of one-dimensional Bin Packing Problem with island parallel grouping genetic algorithms. *Computers & Industrial Engineering* **2014**, *75*, 176–186. https://doi.org/https://doi.org/10.1016/j.cie.2014.06.002.

42. Wilcox, D.; McNabb, A.; Seppi, K. Solving virtual machine packing with a Reordering Grouping Genetic Algorithm. In Proceedings of the 2011 IEEE Congress of Evolutionary Computation (CEC), 2011, pp. 362–369. https://doi.org/10.1109/CEC.2011.5949641.
43. Ozcan, S.O.; Dokeroglu, T.; Cosar, A.; Yazici, A. A Novel Grouping Genetic Algorithm for the One-Dimensional Bin Packing Problem on GPU. In Proceedings of the Computer and Information Sciences; Czachórski, T.; Gelenbe, E.; Grochla, K.; Lent, R., Eds., Cham, 2016; pp. 52–60.
44. Singh, A.; Gupta, A.K. Two heuristics for the one-dimensional bin-packing problem. *OR Spectr.* **2007**, *29*, 765–781. https://doi.org/https://doi.org/10.1007/s00291-006-0071-2.
45. Kucukyilmaz, T.; Kiziloz, H.E. Cooperative parallel grouping genetic algorithm for the one-dimensional bin packing problem. *Computers & Industrial Engineering* **2018**, *125*, 157–170. https://doi.org/https://doi.org/10.1016/j.cie.2018.08.021.
46. Rivera, G.; Cisneros, L.; Sánchez-Solís, P.; Rangel-Valdez, N.; Rodas-Osollo, J. Genetic Algorithm for Scheduling Optimization Considering Heterogeneous Containers: A Real-World Case Study. *Axioms* **2020**, *9*. https://doi.org/10.3390/axioms9010027.
47. Tan, B.; Ma, H.; Mei, Y. A Group Genetic Algorithm for Resource Allocation in Container-Based Clouds. In Proceedings of the Evolutionary Computation in Combinatorial Optimization; Paquete, L.; Zarges, C., Eds., Cham, 2020; pp. 180–196.
48. Carmona-Arroyo, G.; Vázquez-Aguirre, J.B.; Quiroz-Castellanos, M. *One-Dimensional Bin Packing Problem: An Experimental Study of Instances Difficulty and Algorithms Performance*; Vol. 940, Springer, Cham, 2021.
49. Scholl, A.; Klein, R.; Jürgens, C. Bison: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem. *Computers & Operations Research* **1997**, *24*, 627–645. https://doi.org/https://doi.org/10.1016/S0305-0548(96)00082-2.
50. Schwerin, P.; Wäscher, G. The bin-packing problem: A problem generator and some numerical experiments with FFD packing and MTP. *International Transactions in Operational Research* **1997**, *4*, 377–389. https://doi.org/https://doi.org/10.1016/S0969-6016(97)00025-7.
51. Wiischer, G.; Gau, T. ORSpe m Heuristics for the Integer One-dimensional Cutting Stock Problem: a computational study. *OR Spektrum* **1996**, *18*, 131–144.
52. Schoenfield, J.E. Fast, exact solution of open bin packing problems without linear programming. *Draft, US Army Space and Missile Defense Command, Huntsville, Alabama, USA* **2002**.
53. Agustín-Blas, L.; Salcedo-Sanz, S.; Jiménez-Fernández, S.; Carro-Calvo, L.; Del Ser, J.; Portilla-Figueras, J. A new grouping genetic algorithm for clustering problems. *Expert Systems with Applications* **2012**, *39*, 9695–9703. https://doi.org/https://doi.org/10.1016/j.eswa.2012.02.149.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.